

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Cache and Memory Hierarchy (Part 2)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Mikko Lipasti

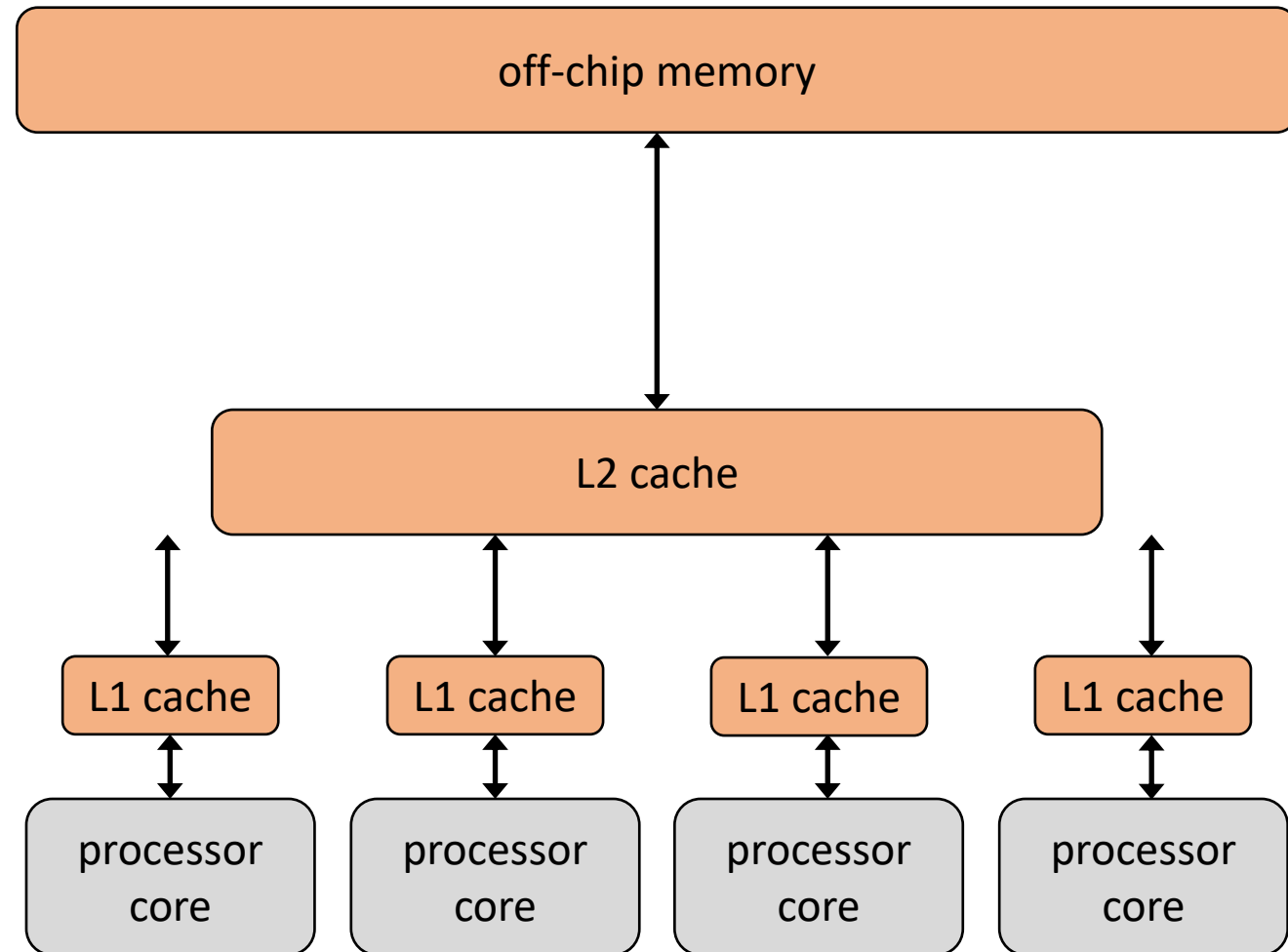
Last Lecture

- Centralized barriers
- Tree barriers
- Shared memory CMPs

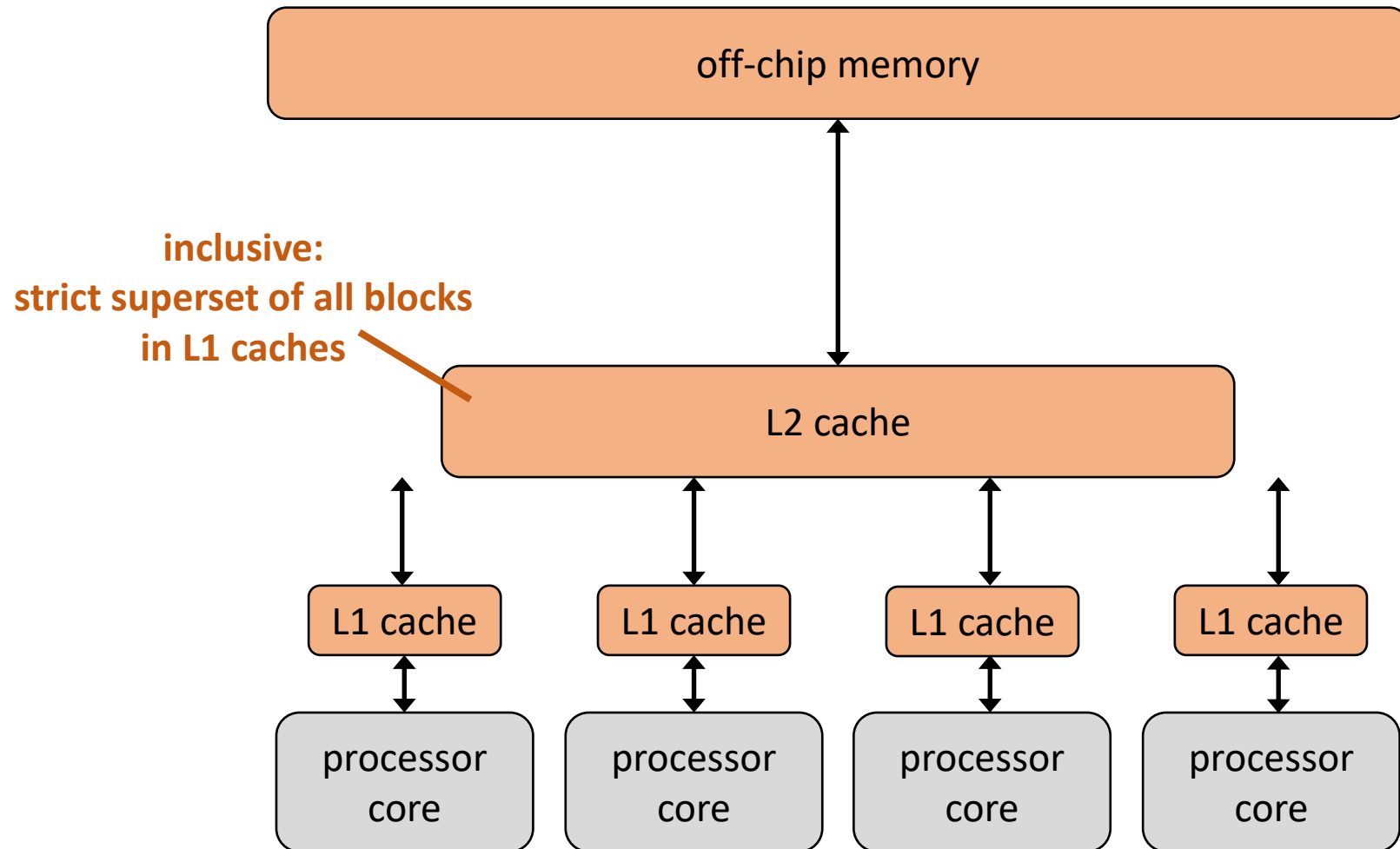
This Lecture

- Cache architecture
- Cache misses
- False sharing
- Parallelism and cache locality

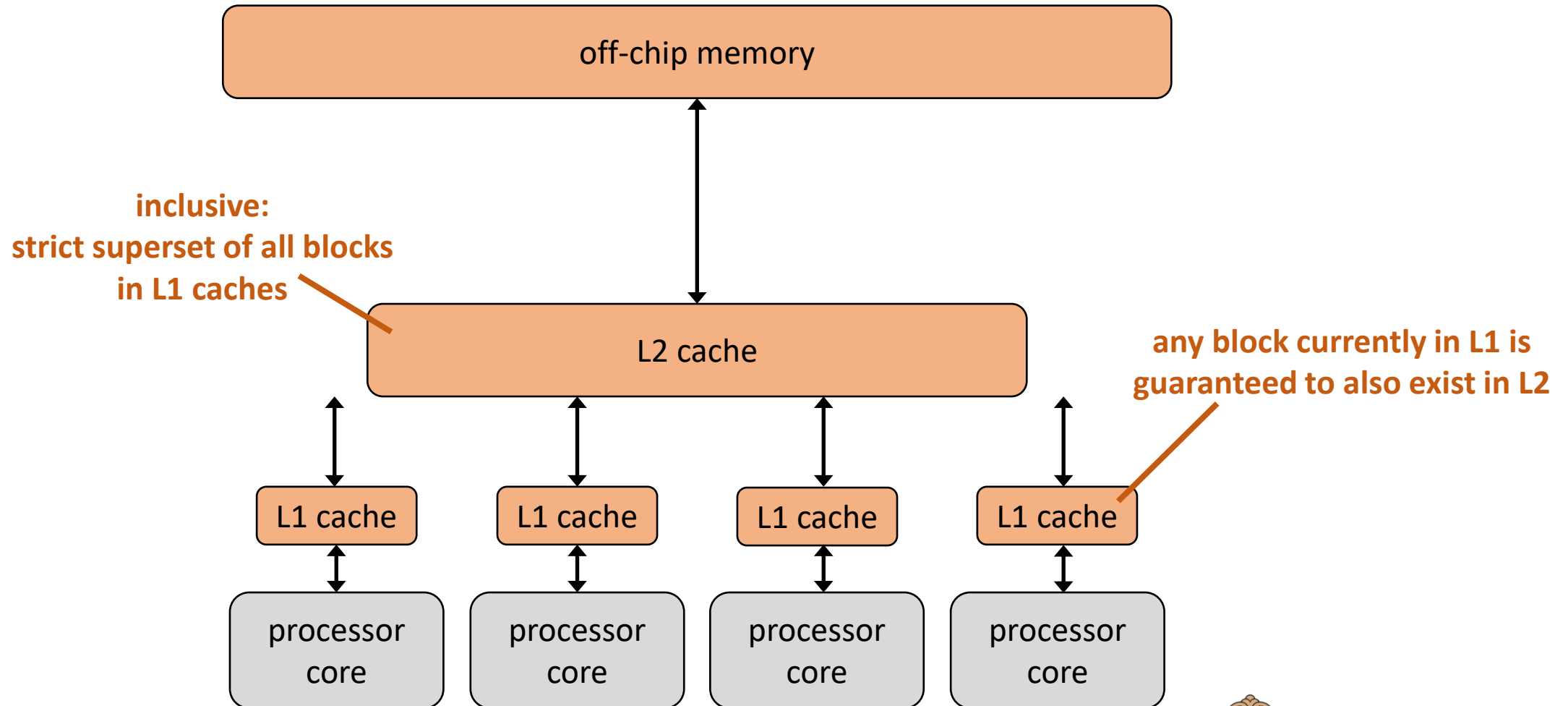
Shared-Memory CMP



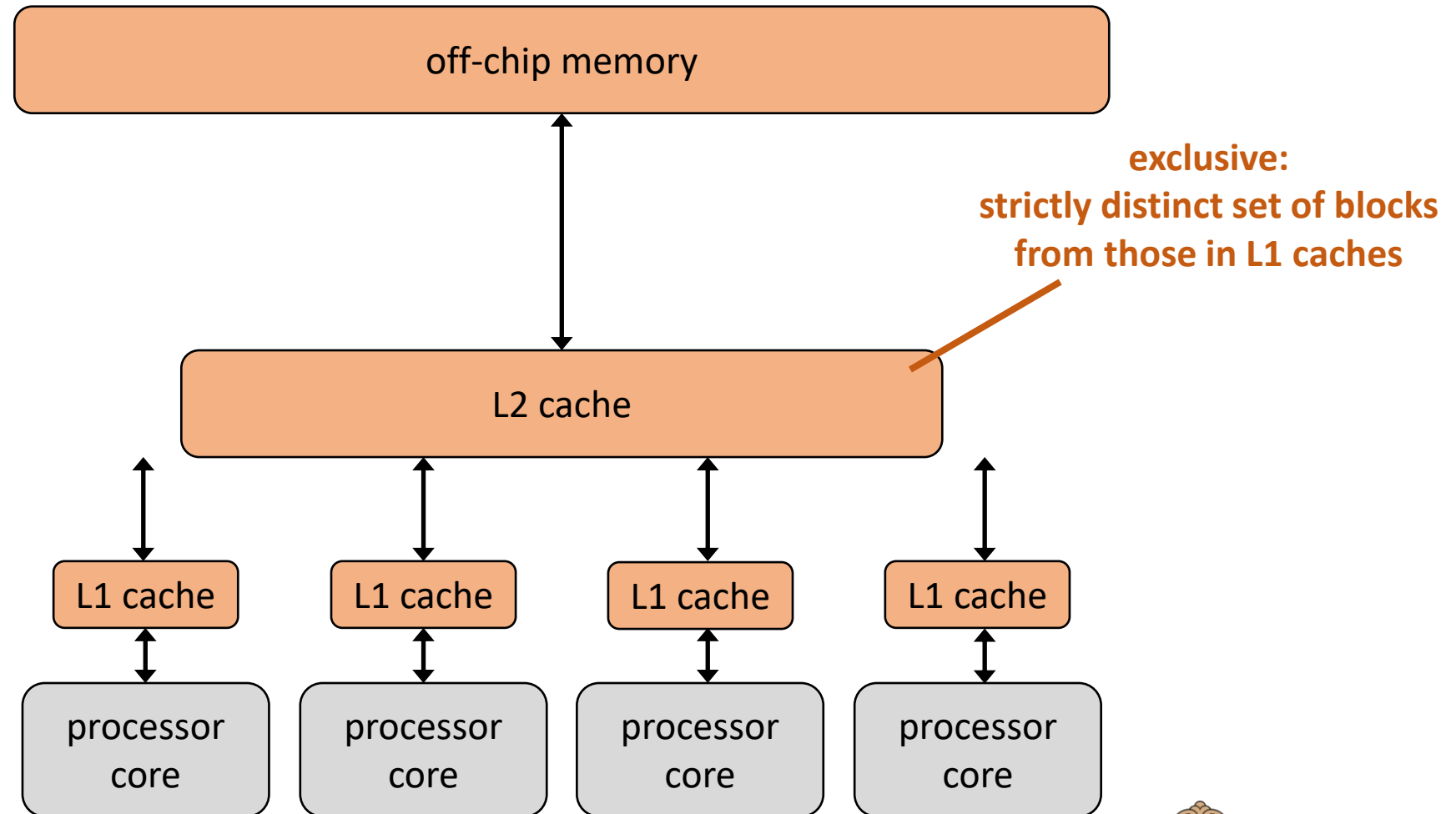
Shared-Memory CMP – Inclusive Cache



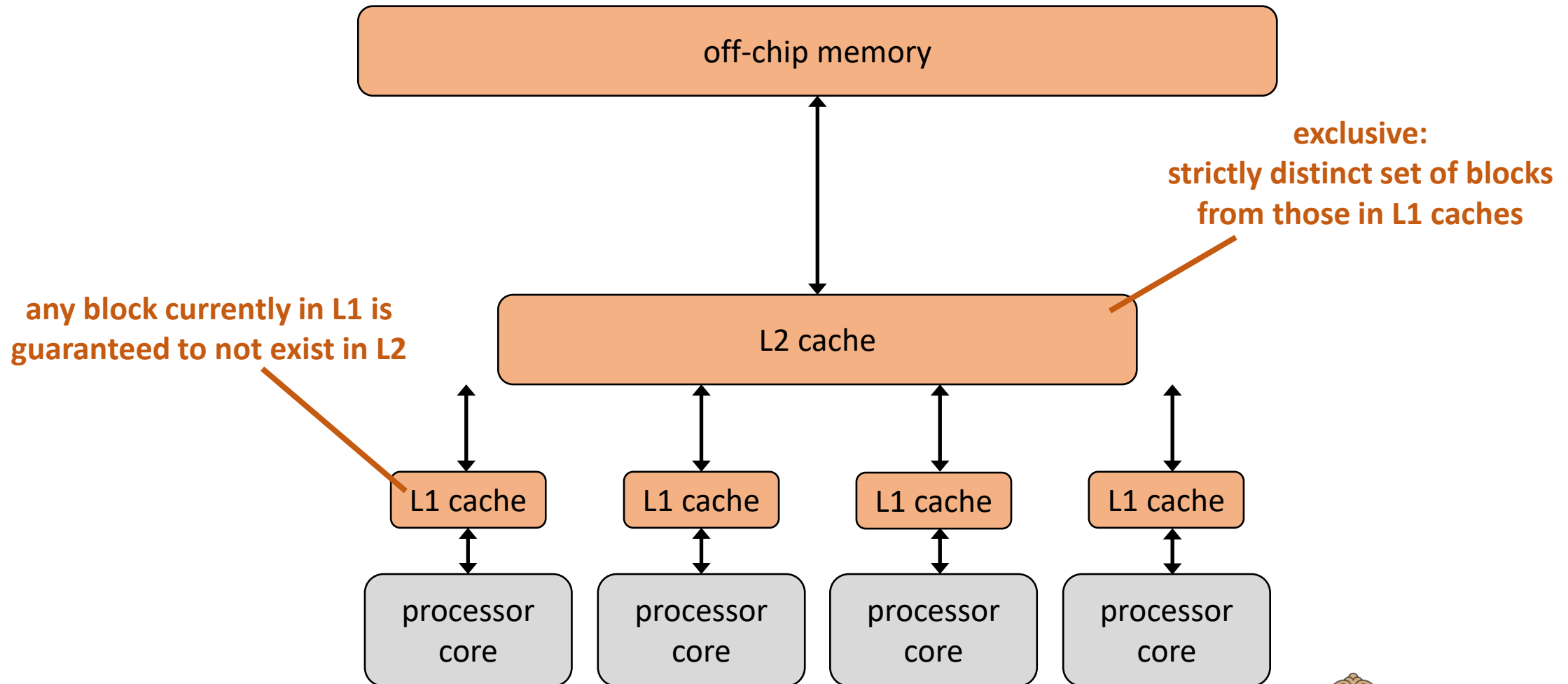
Shared-Memory CMP – Inclusive Cache



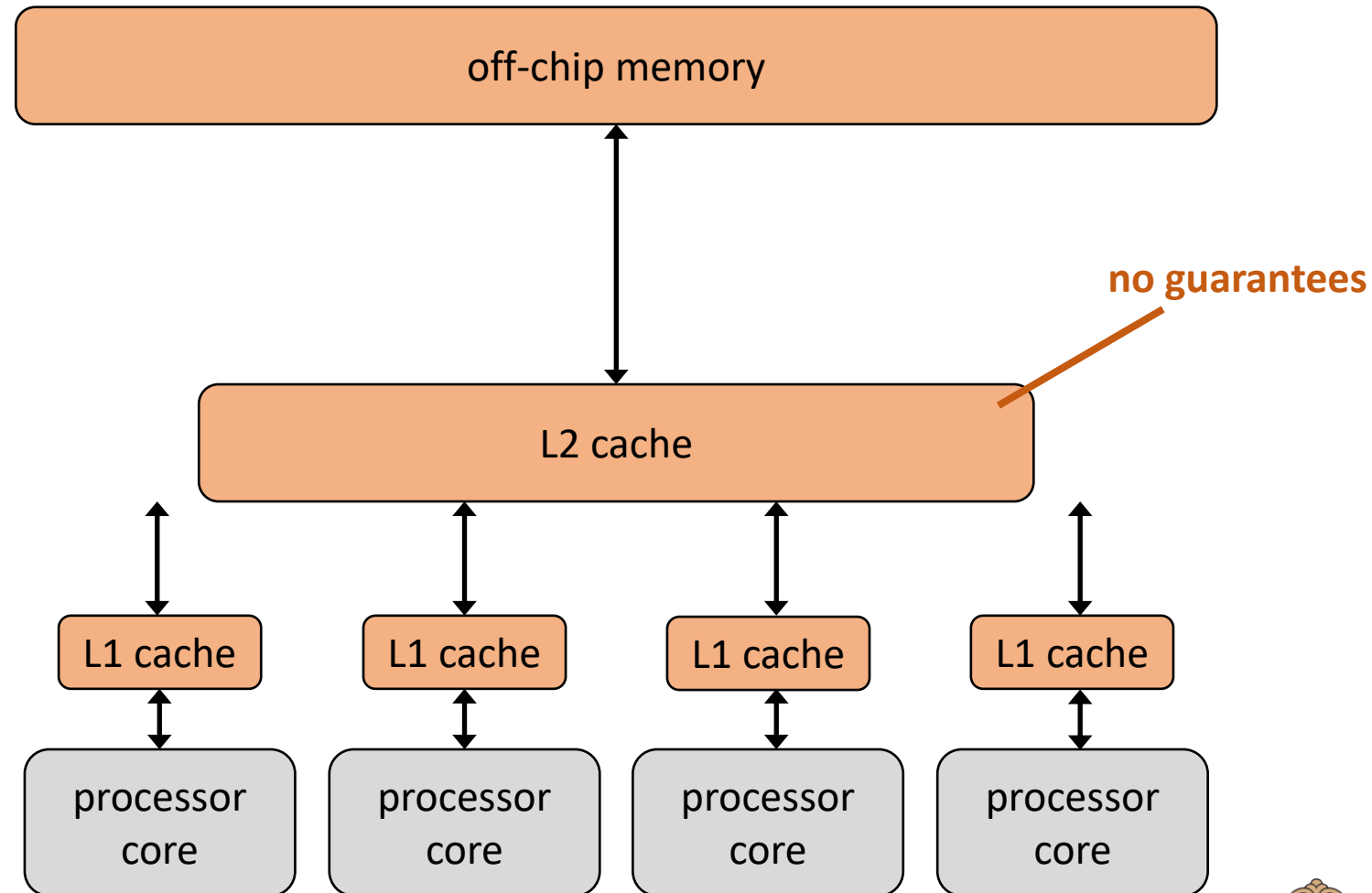
Shared-Memory CMP – Exclusive Cache



Shared-Memory CMP – Exclusive Cache



Shared-Memory CMP – Non-Inclusive/-Exclusive



Inclusive and Exclusive Caches

Inclusive Cache

Exclusive Cache

VS

Inclusive and Exclusive Caches

Inclusive Cache

- Lower effective capacity (has redundant blocks among levels)
- More eviction traffic downstream (e.g., if L2 evicts block, need to force evict block from L1 caches as well)

VS

Exclusive Cache

- More overhead of checking if block exists for coherence (need to check all private levels)
- More eviction traffic upstream (e.g., if L1 evicts block, send data back to L2)

Inclusive and Exclusive Caches

What if working set is a bit larger than L1 capacity?

Inclusive L2 Cache

Exclusive L2 Cache

VS



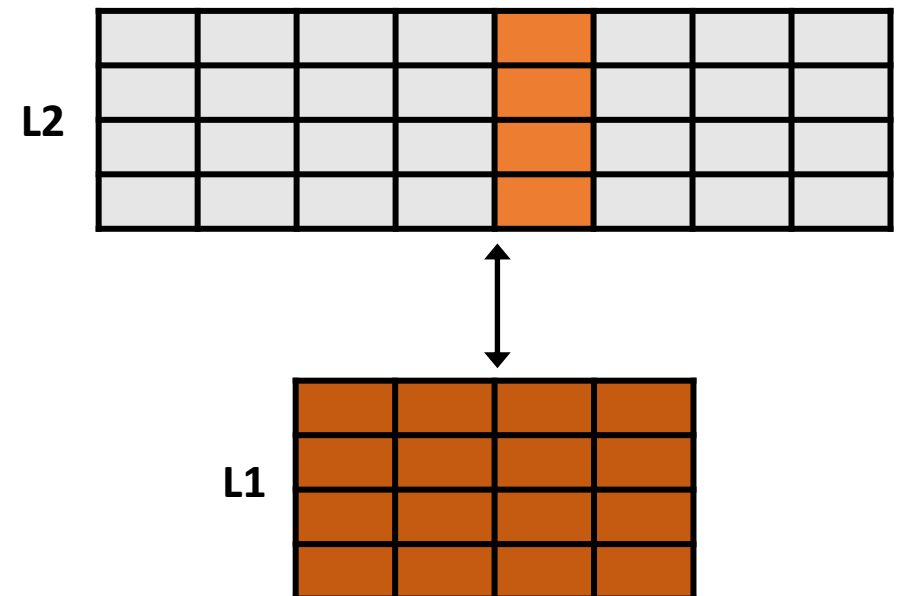
Inclusive and Exclusive Caches

What if working set is a bit larger than L1 capacity?

Inclusive L2 Cache

VS

Exclusive L2 Cache



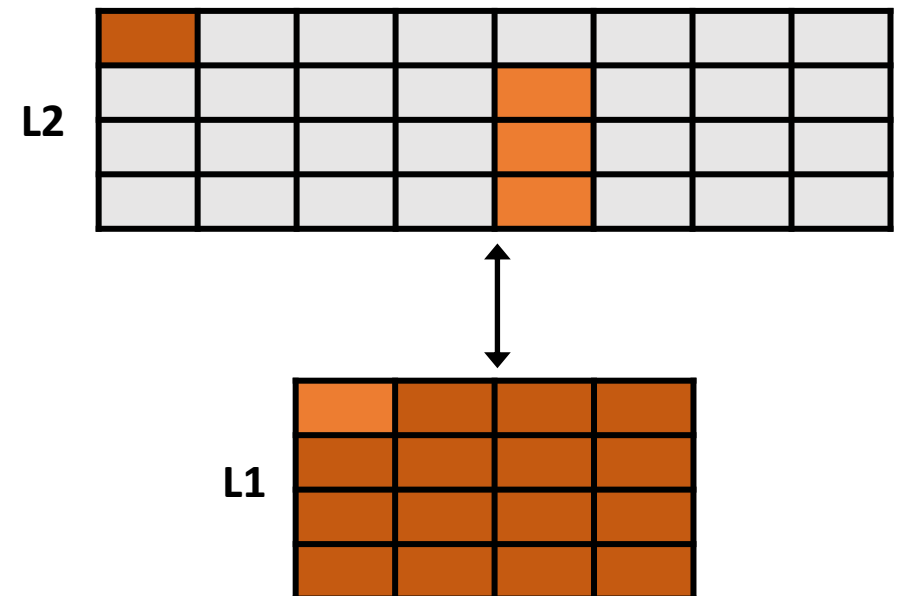
Inclusive and Exclusive Caches

What if working set is a bit larger than L1 capacity?

Inclusive L2 Cache

VS

Exclusive L2 Cache



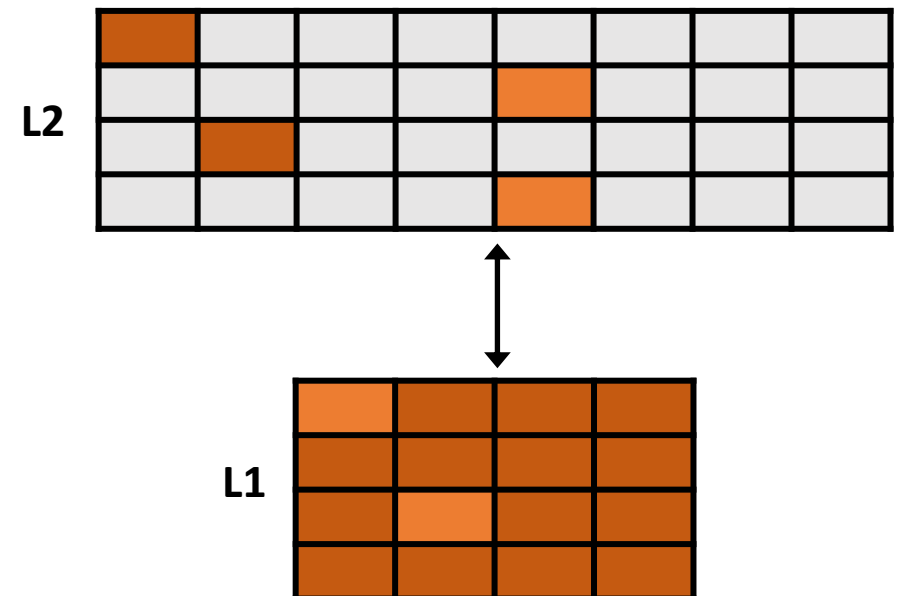
Inclusive and Exclusive Caches

What if working set is a bit larger than L1 capacity?

Inclusive L2 Cache

Exclusive L2 Cache

VS



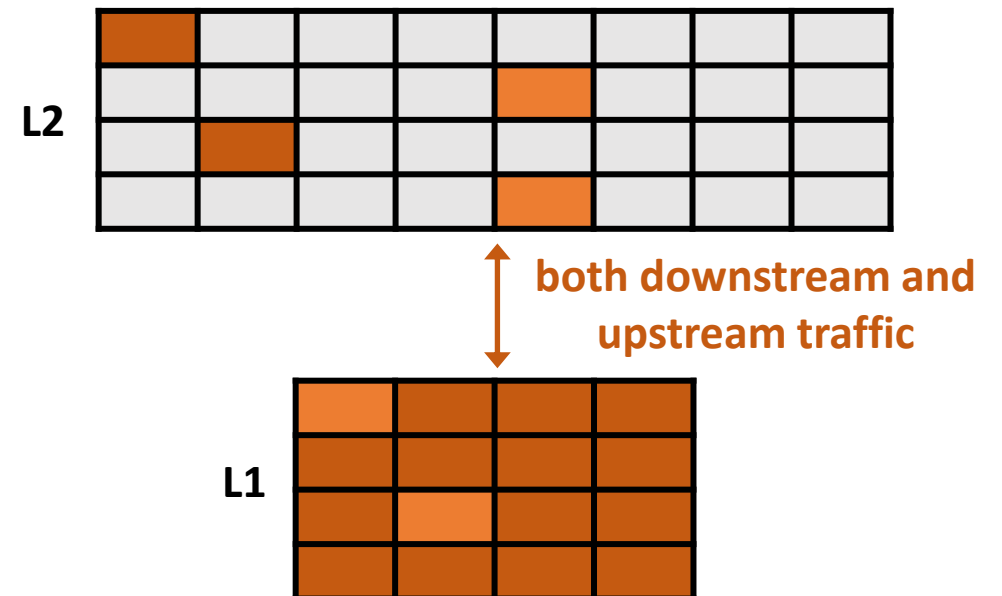
Inclusive and Exclusive Caches

What if working set is a bit larger than L1 capacity?

Inclusive L2 Cache

VS

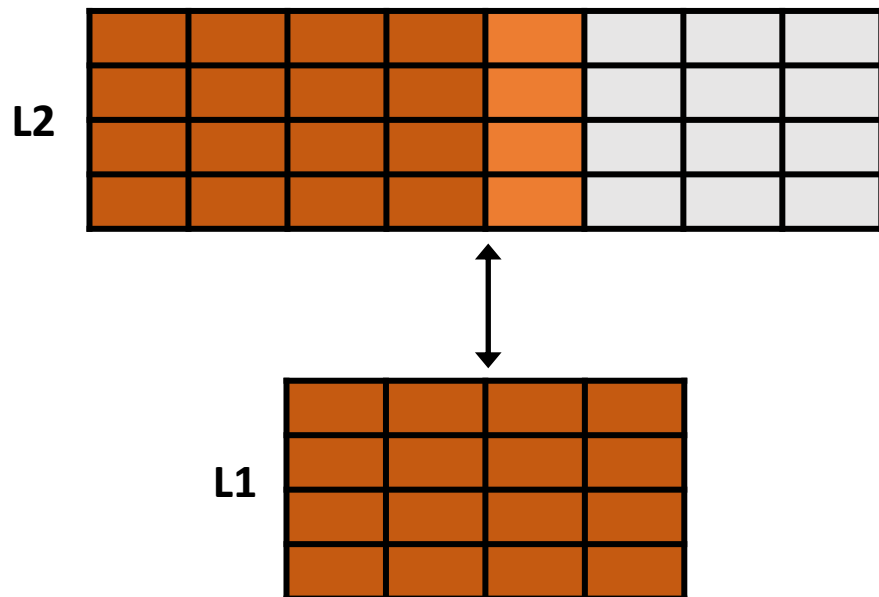
Exclusive L2 Cache



Inclusive and Exclusive Caches

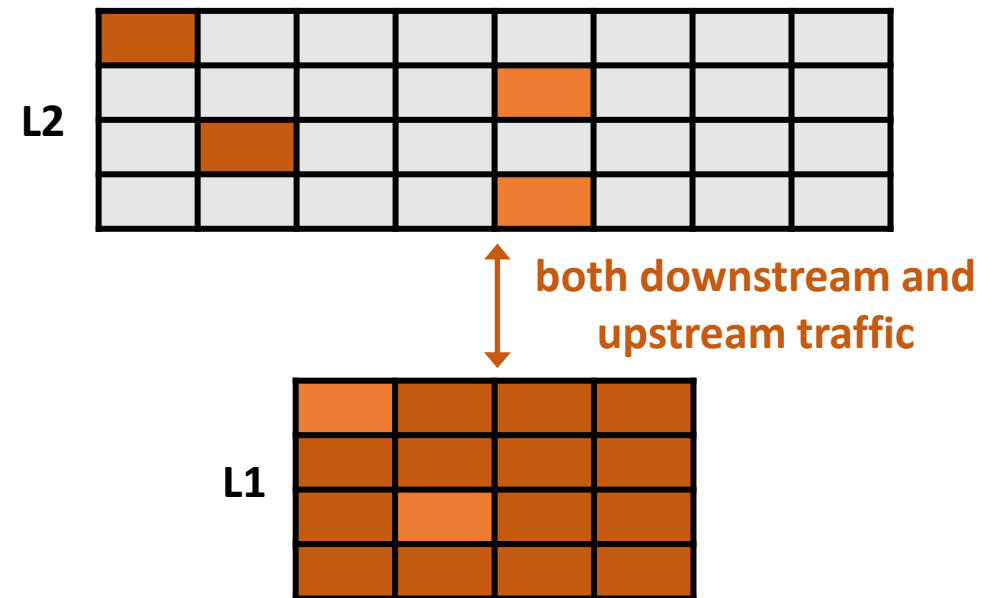
What if working set is a bit larger than L1 capacity?

Inclusive L2 Cache



VS

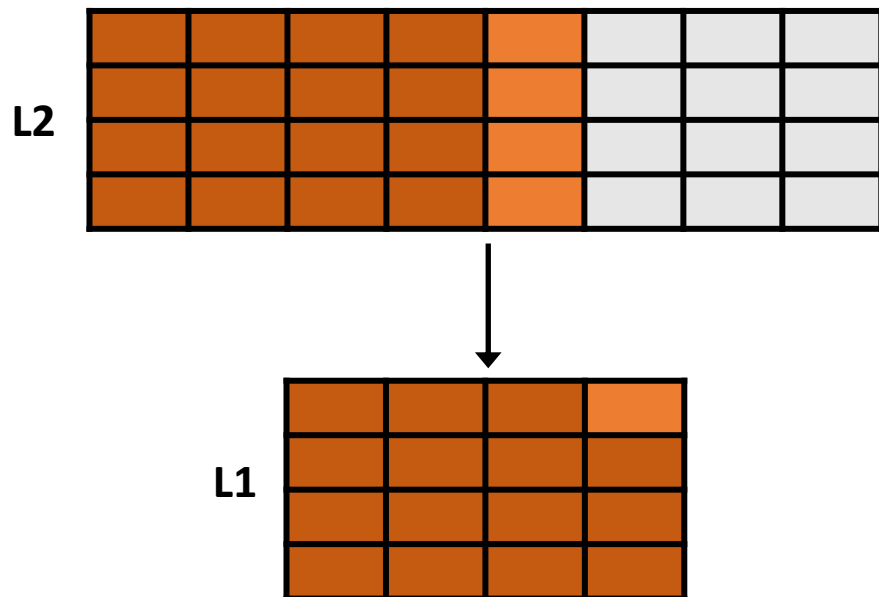
Exclusive L2 Cache



Inclusive and Exclusive Caches

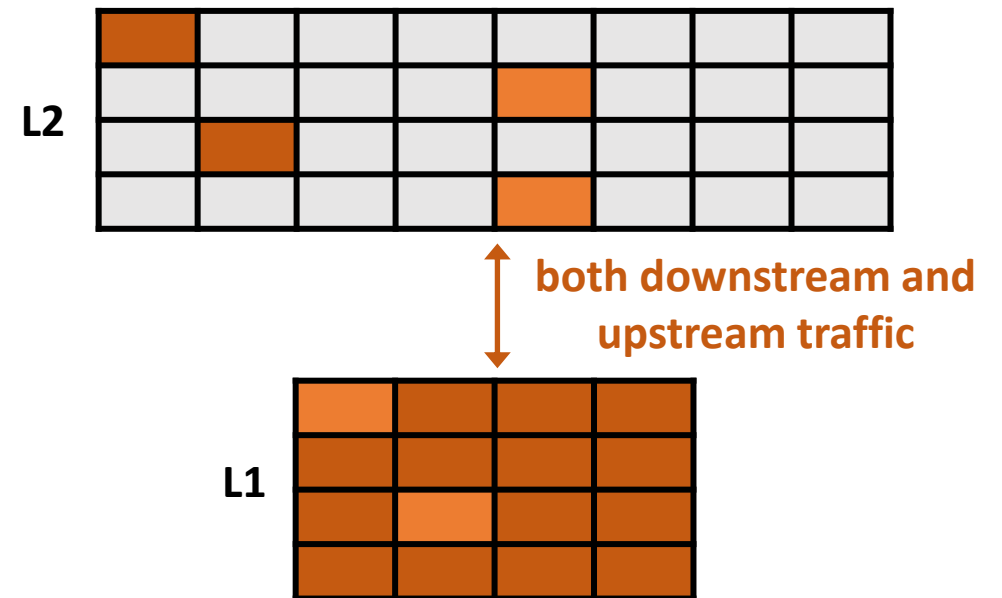
What if working set is a bit larger than L1 capacity?

Inclusive L2 Cache



VS

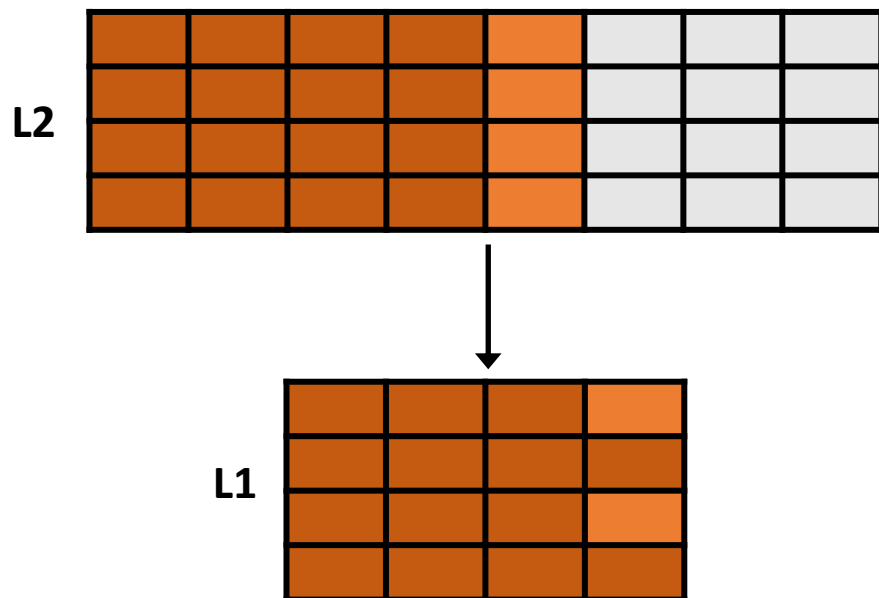
Exclusive L2 Cache



Inclusive and Exclusive Caches

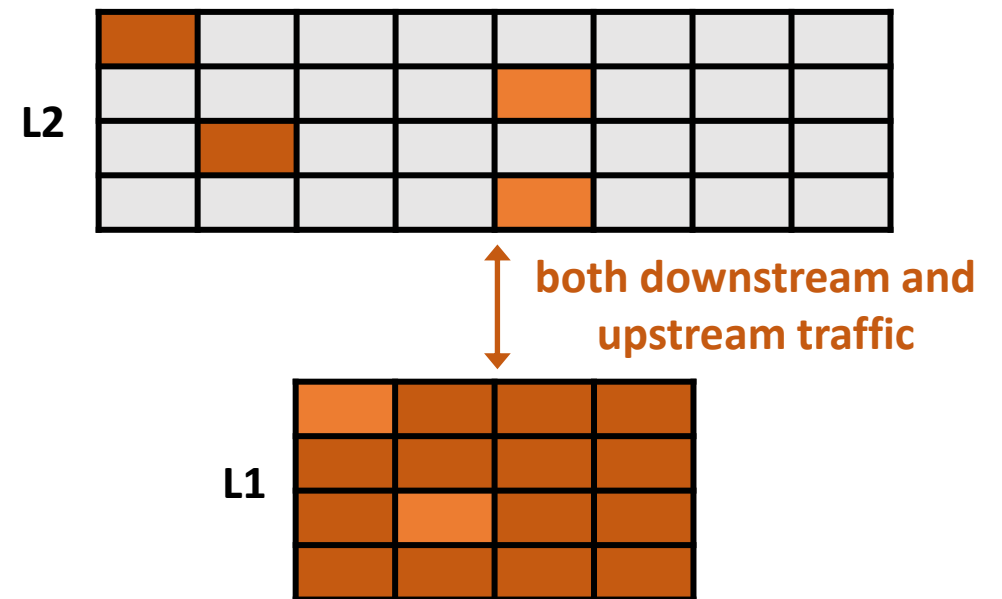
What if working set is a bit larger than L1 capacity?

Inclusive L2 Cache



VS

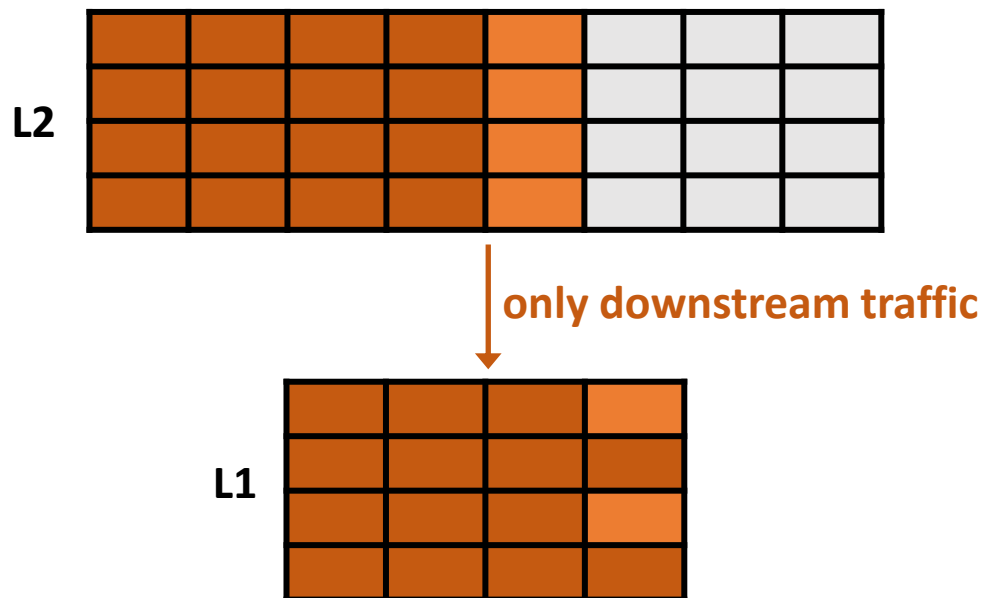
Exclusive L2 Cache



Inclusive and Exclusive Caches

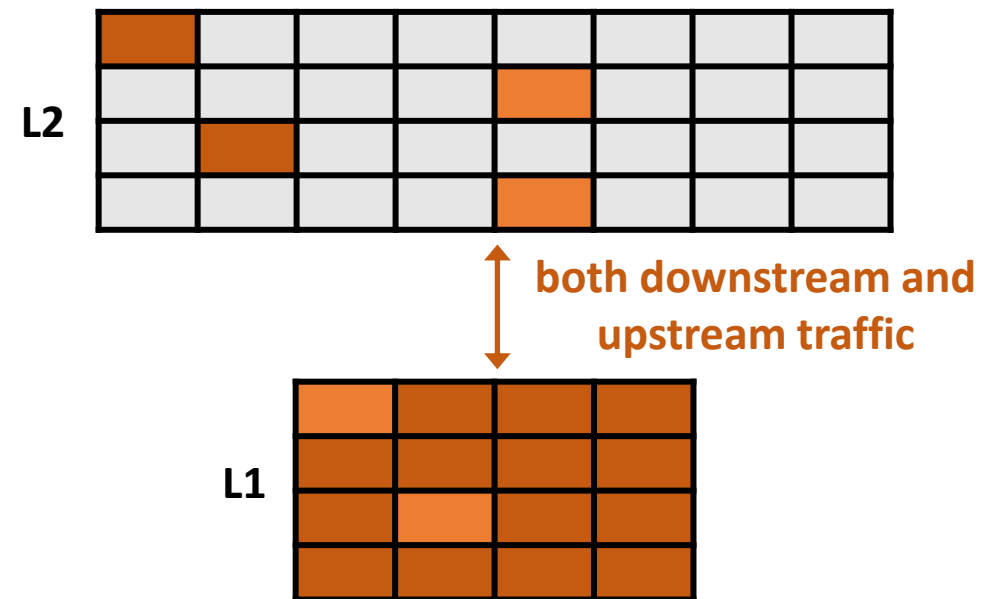
What if working set is a bit larger than L1 capacity?

Inclusive L2 Cache



VS

Exclusive L2 Cache



Inclusive and Exclusive Caches

What if working set is about as large as aggregate L1+L2 capacity?

Inclusive L2 Cache

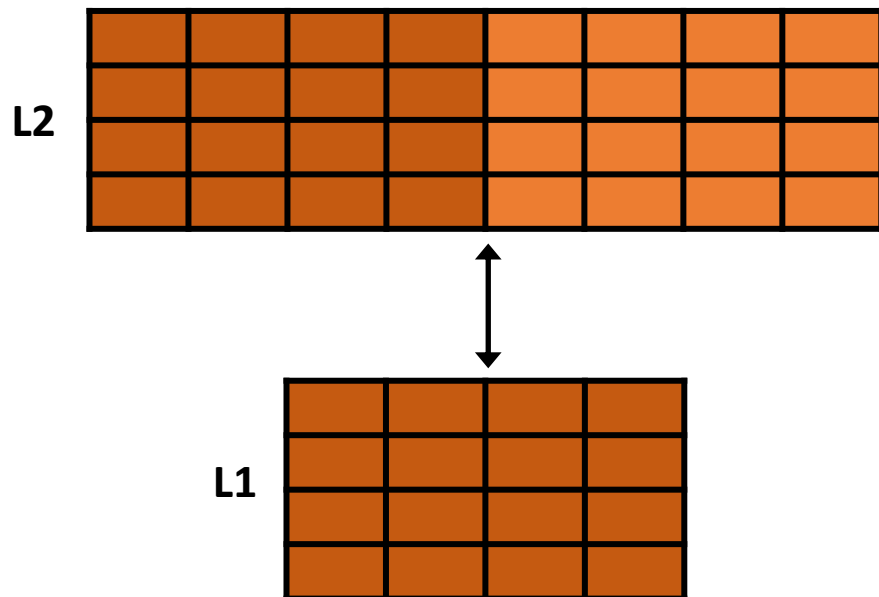
Exclusive L2 Cache

VS

Inclusive and Exclusive Caches

What if working set is about as large as aggregate L1+L2 capacity?

Inclusive L2 Cache



VS

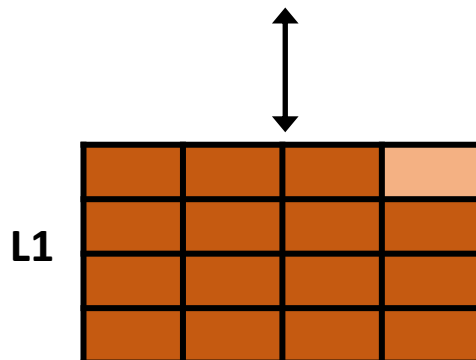
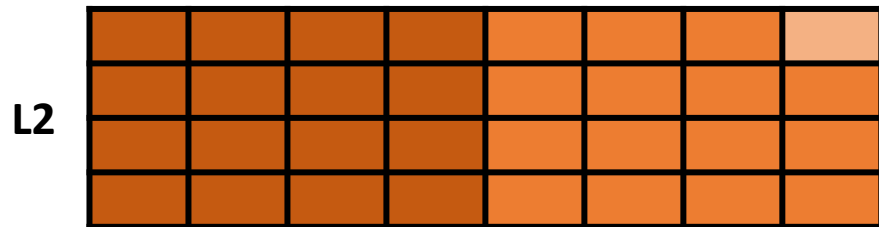
Exclusive L2 Cache

Inclusive and Exclusive Caches

What if working set is about as large as aggregate L1+L2 capacity?

Inclusive L2 Cache

need to get more blocks from off-chip memory



VS

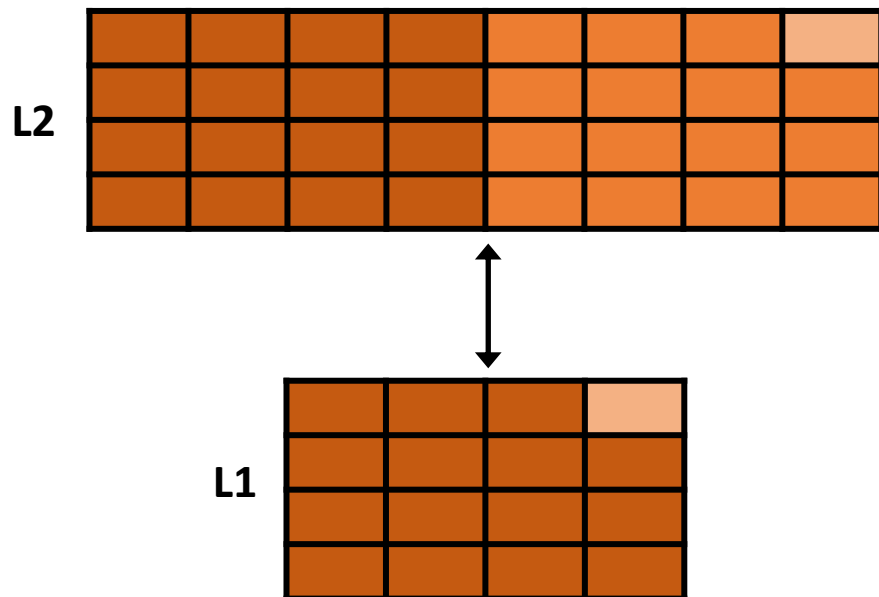
Exclusive L2 Cache

Inclusive and Exclusive Caches

What if working set is about as large as aggregate L1+L2 capacity?

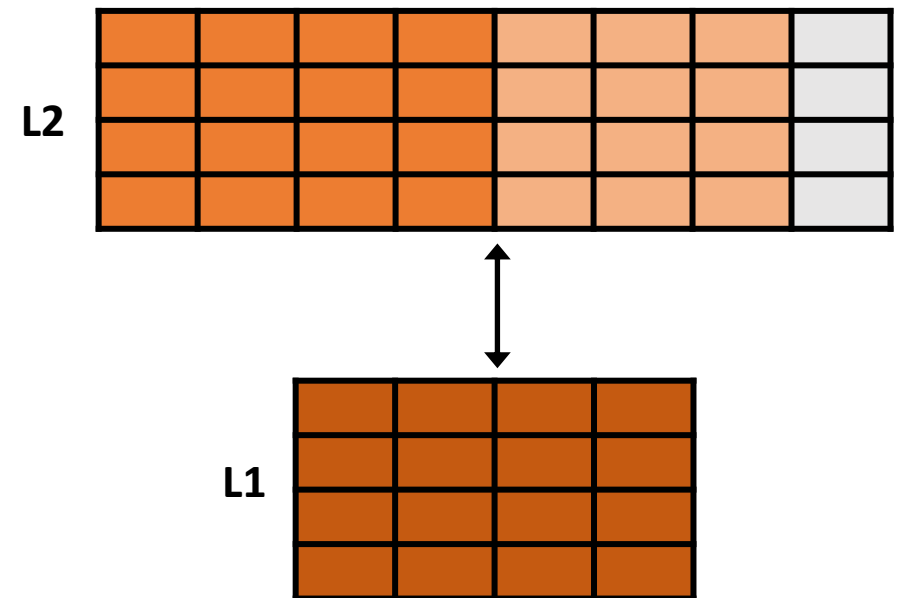
Inclusive L2 Cache

need to get more blocks from off-chip memory



VS

Exclusive L2 Cache

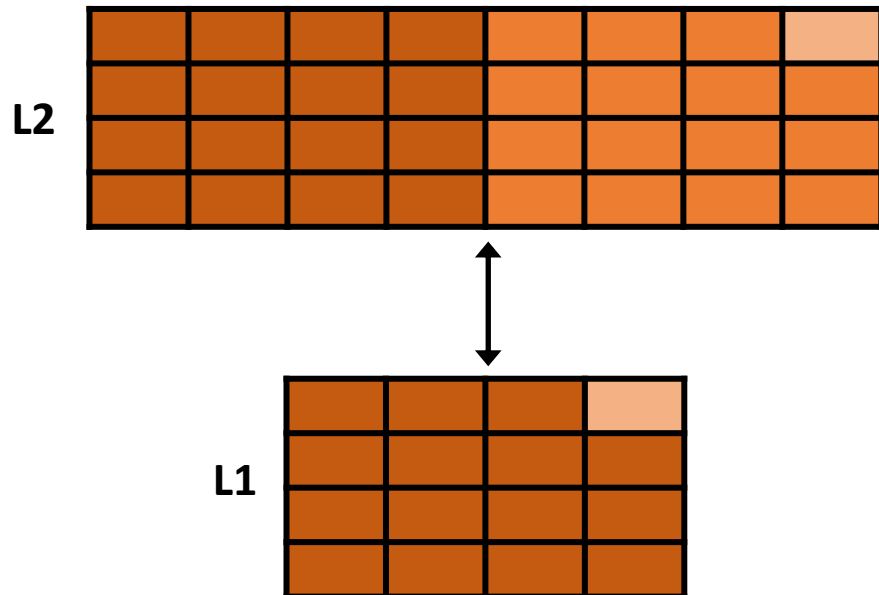


Inclusive and Exclusive Caches

What if working set is about as large as aggregate L1+L2 capacity?

Inclusive L2 Cache

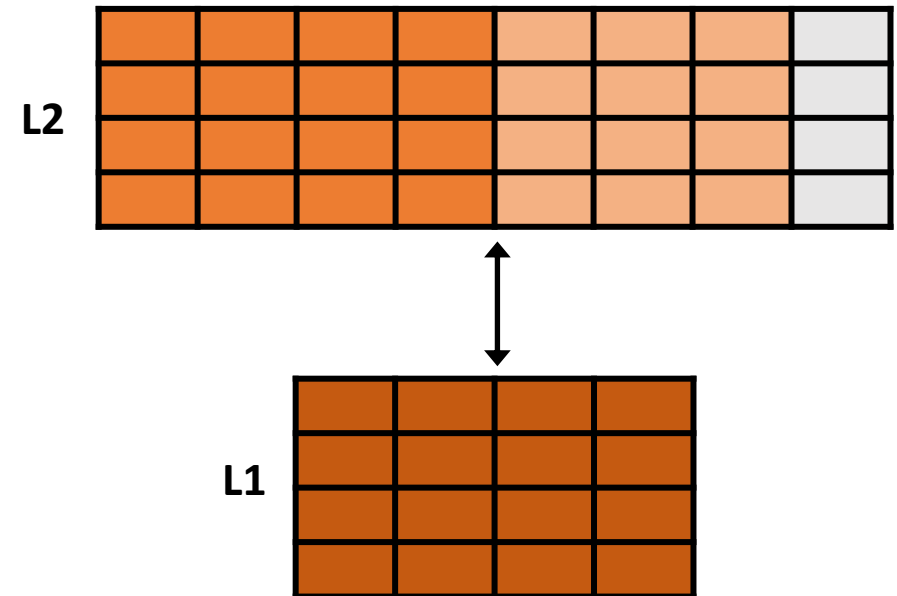
need to get more blocks from off-chip memory



VS

Exclusive L2 Cache

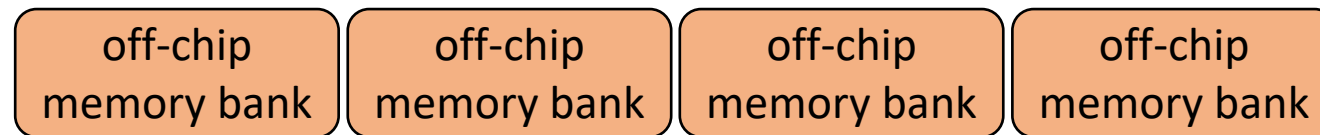
no need to get blocks from off-chip memory



NUMA and NUCA

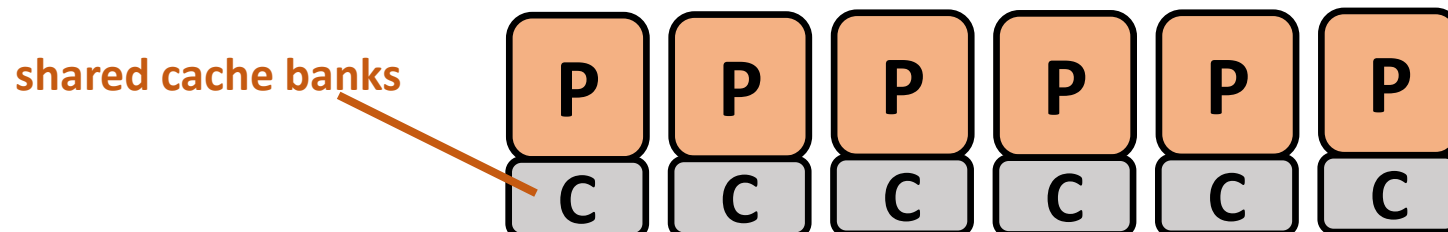
Non-Uniform Memory Access (NUMA):

Memory requests for different addresses can take different amounts of time (e.g., interleaved memory).



Non-Uniform Cache Access (NUCA):

Cache requests for different addresses can take different amounts of time (e.g., distributed cache).



Cache Architecture

Placement:

Where can a block of memory go?

Replacement:

How do we make space for new blocks?

Write Policy:

How do we propagate modifications?

Cache Architecture – Placement

Since physical address range exceeds cache capacity, map physical memory addresses into finite set of cache storage locations.

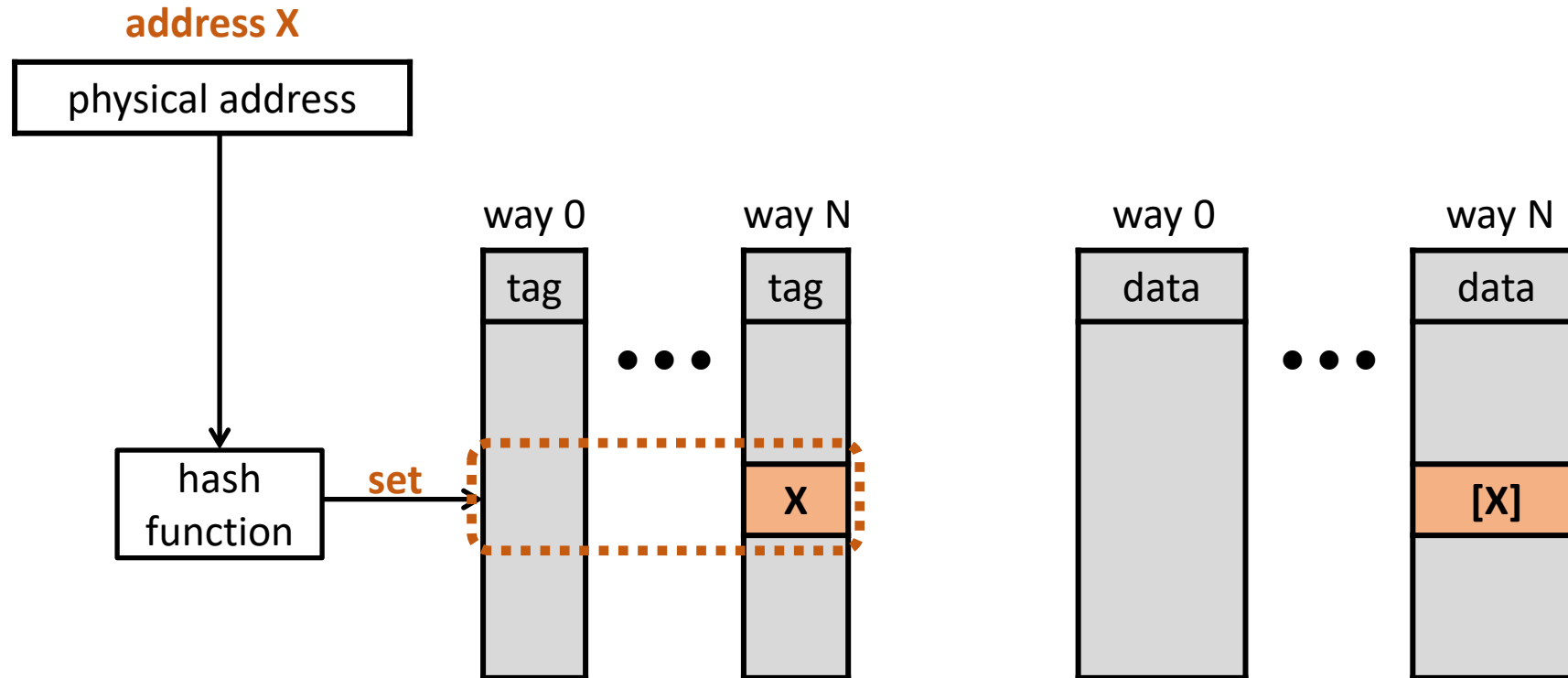
Cache Set: Which group of possible storage locations can a block be placed?

Cache Associativity: How many possible locations can a block be placed (i.e., ways)?

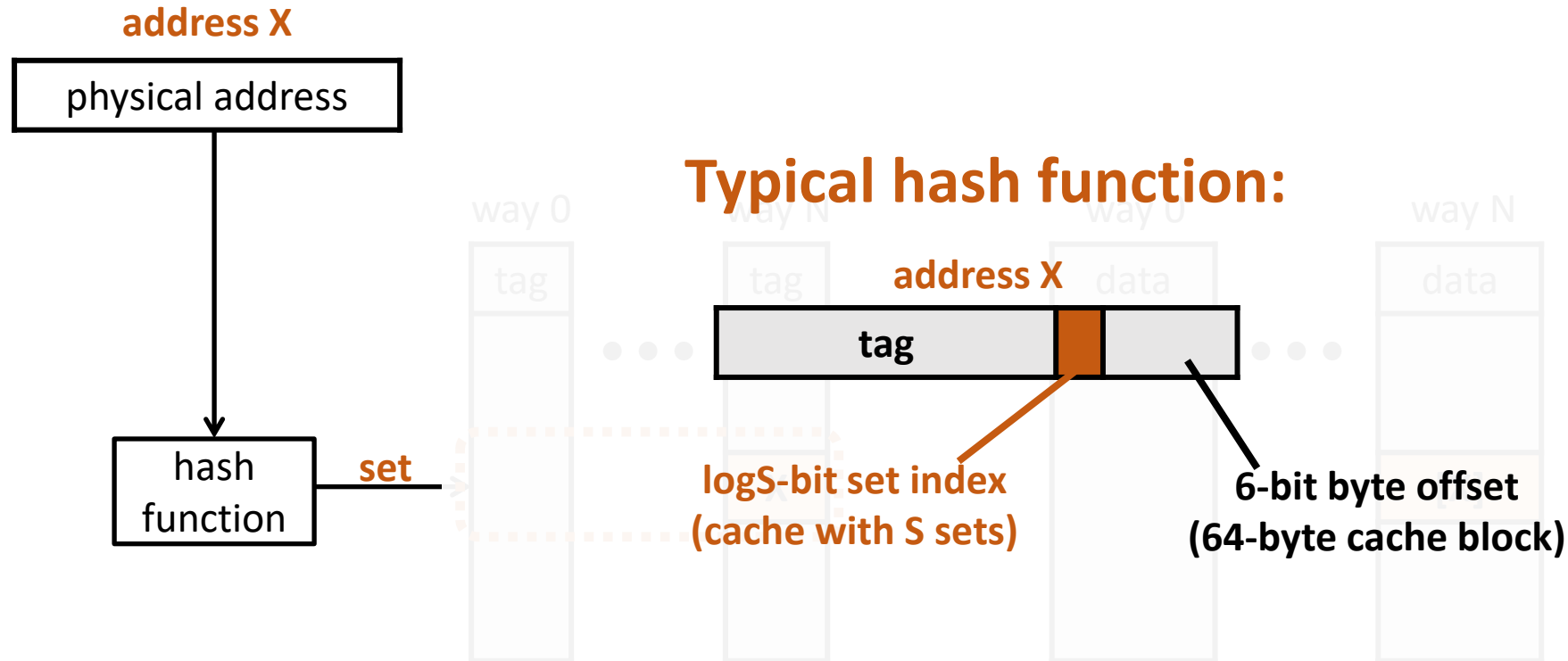
Direct-Mapped: 1-way associative

Fully-Associative: N-way associative, where N is the cache capacity (in blocks)

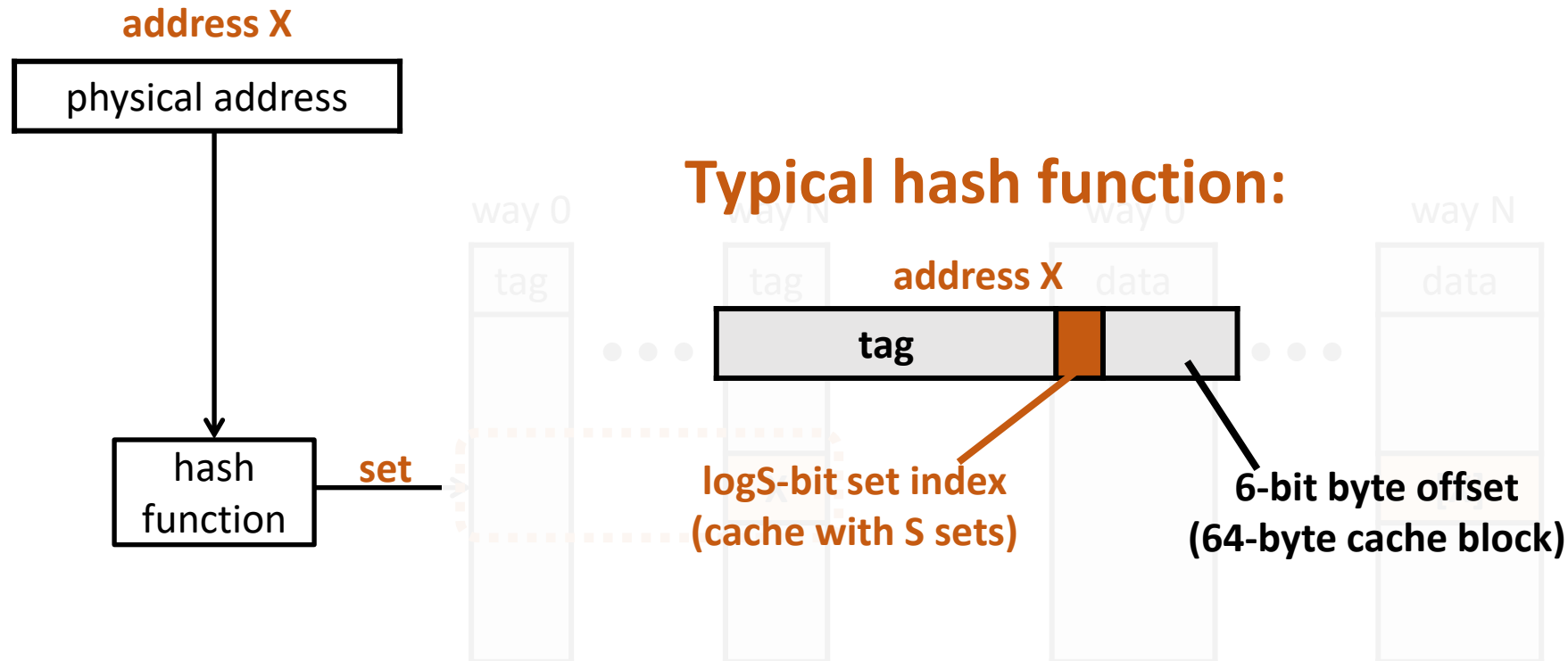
Cache Architecture – Placement



Cache Architecture – Placement

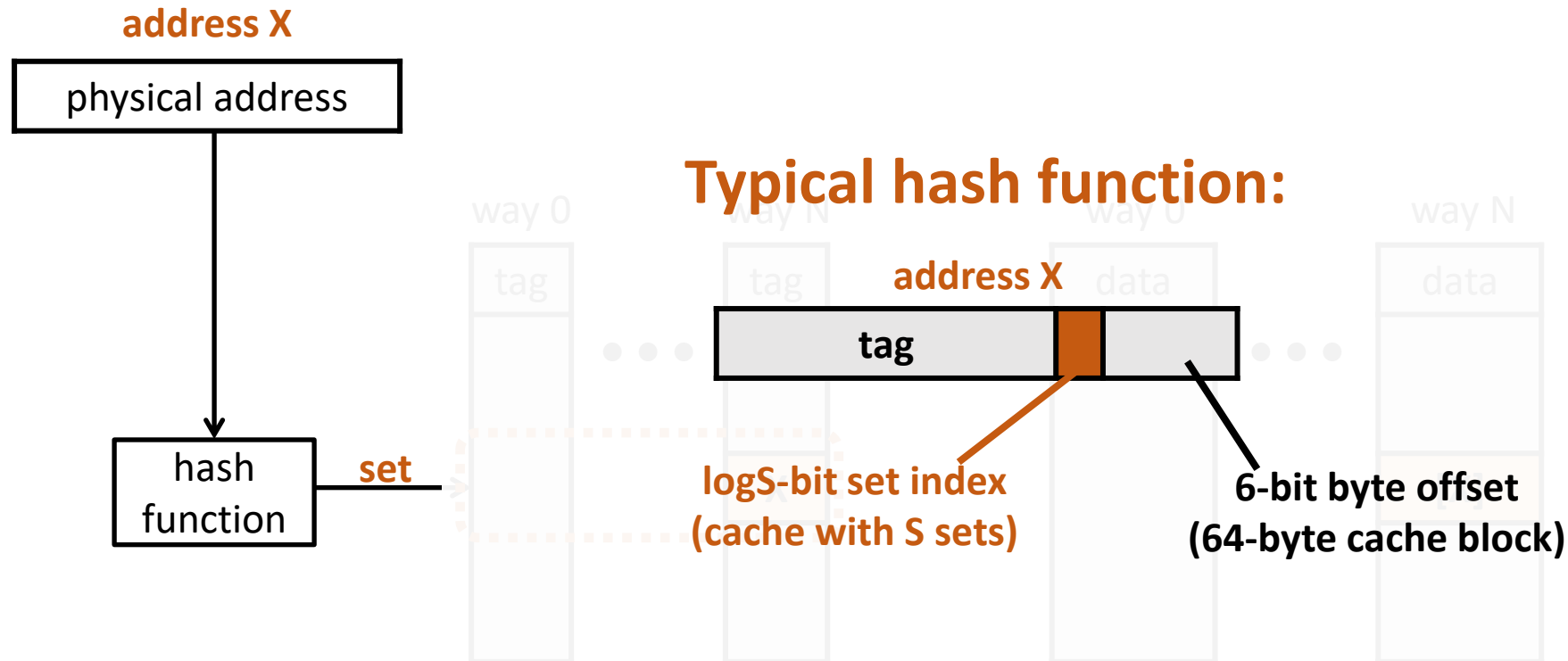


Cache Architecture – Placement



Given 32-bit addresses and 64-byte blocks, what is the minimum number of tag bits needed in a 16-way associative 128KB cache?

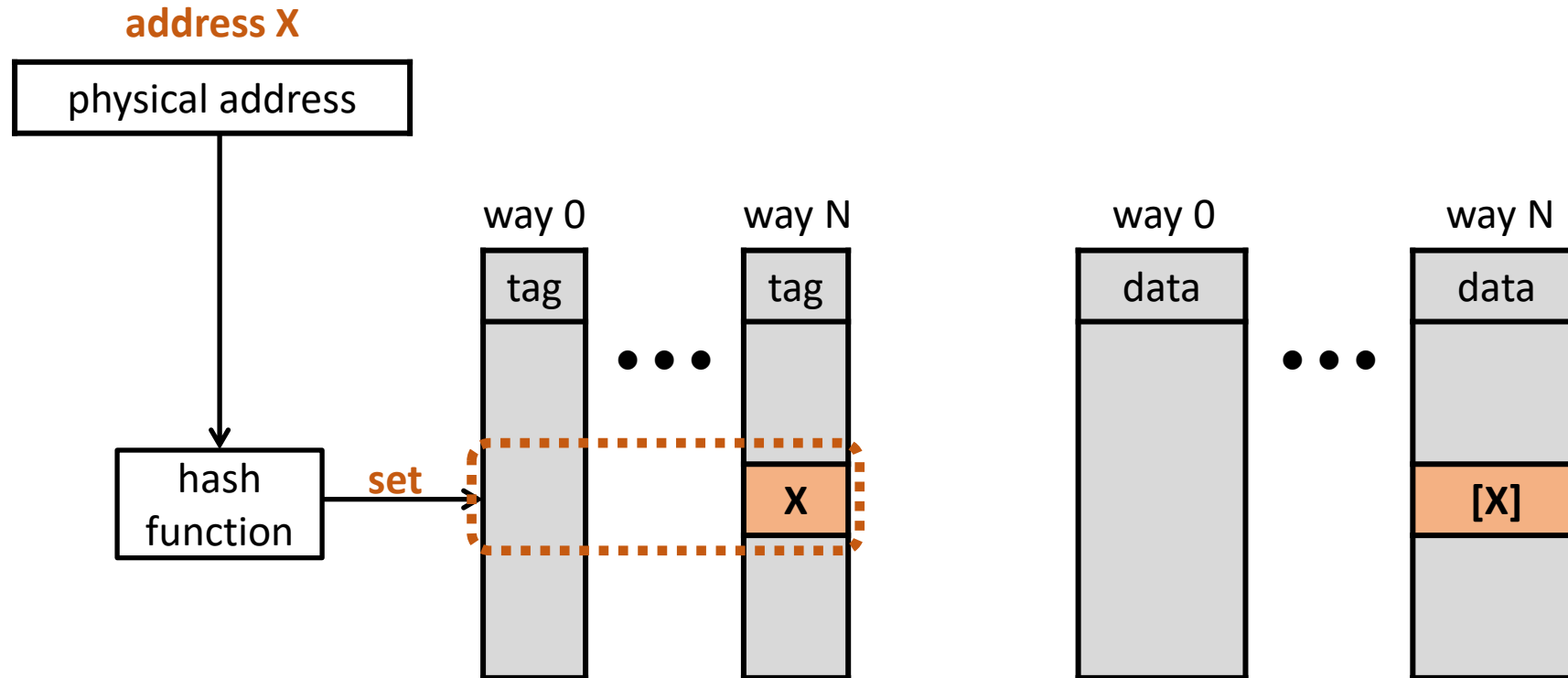
Cache Architecture – Placement



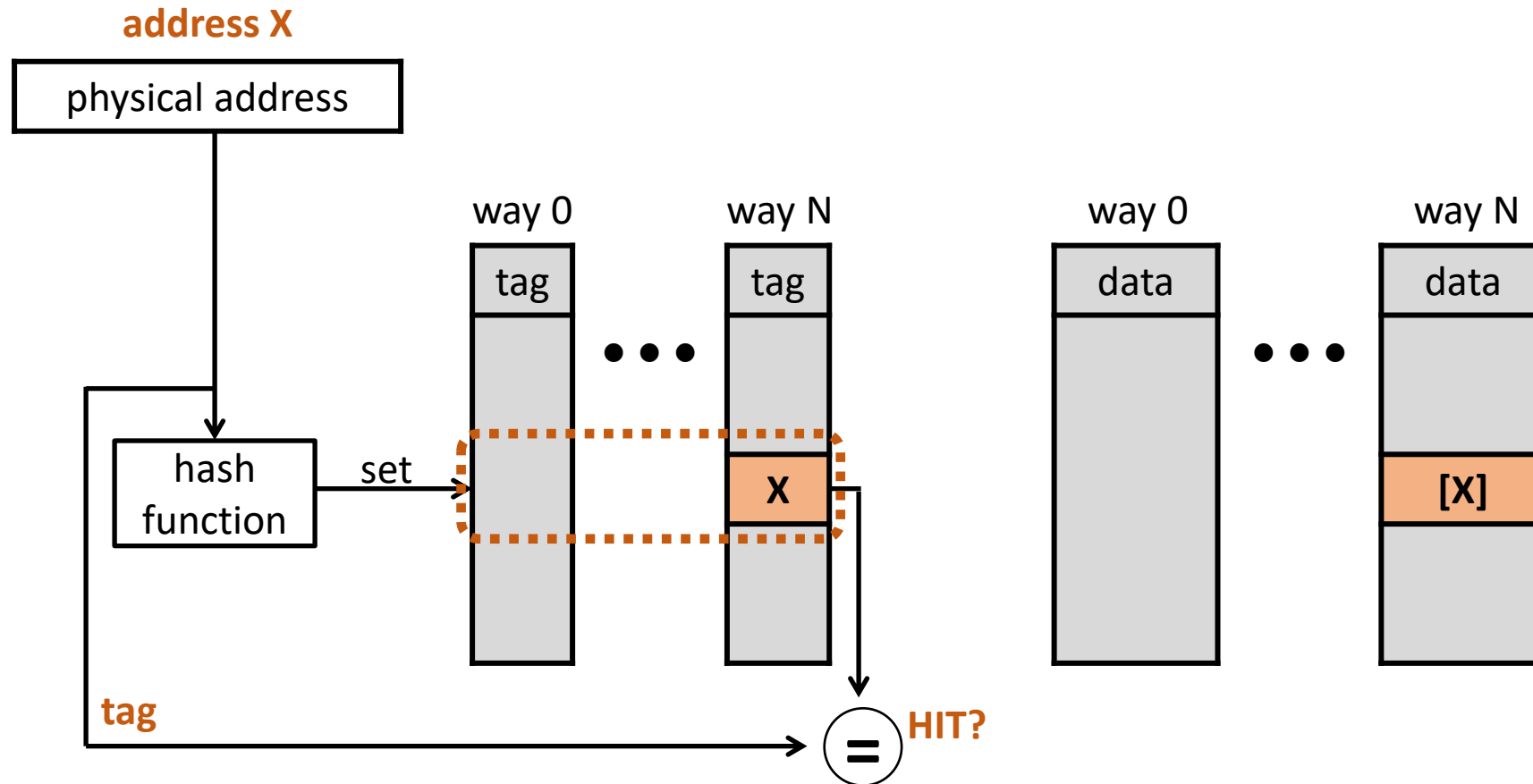
Given 32-bit addresses and 64-byte blocks, what is the minimum number of tag bits needed in a 16-way associative 128KB cache?

- $(128\text{KB} / 64\text{B per block}) / 16 \text{ ways} = 128 \text{ sets}$
- $32 \text{ bits} - \log(64\text{B per block}) - \log(128 \text{ sets}) = 19 \text{ tag bits}$

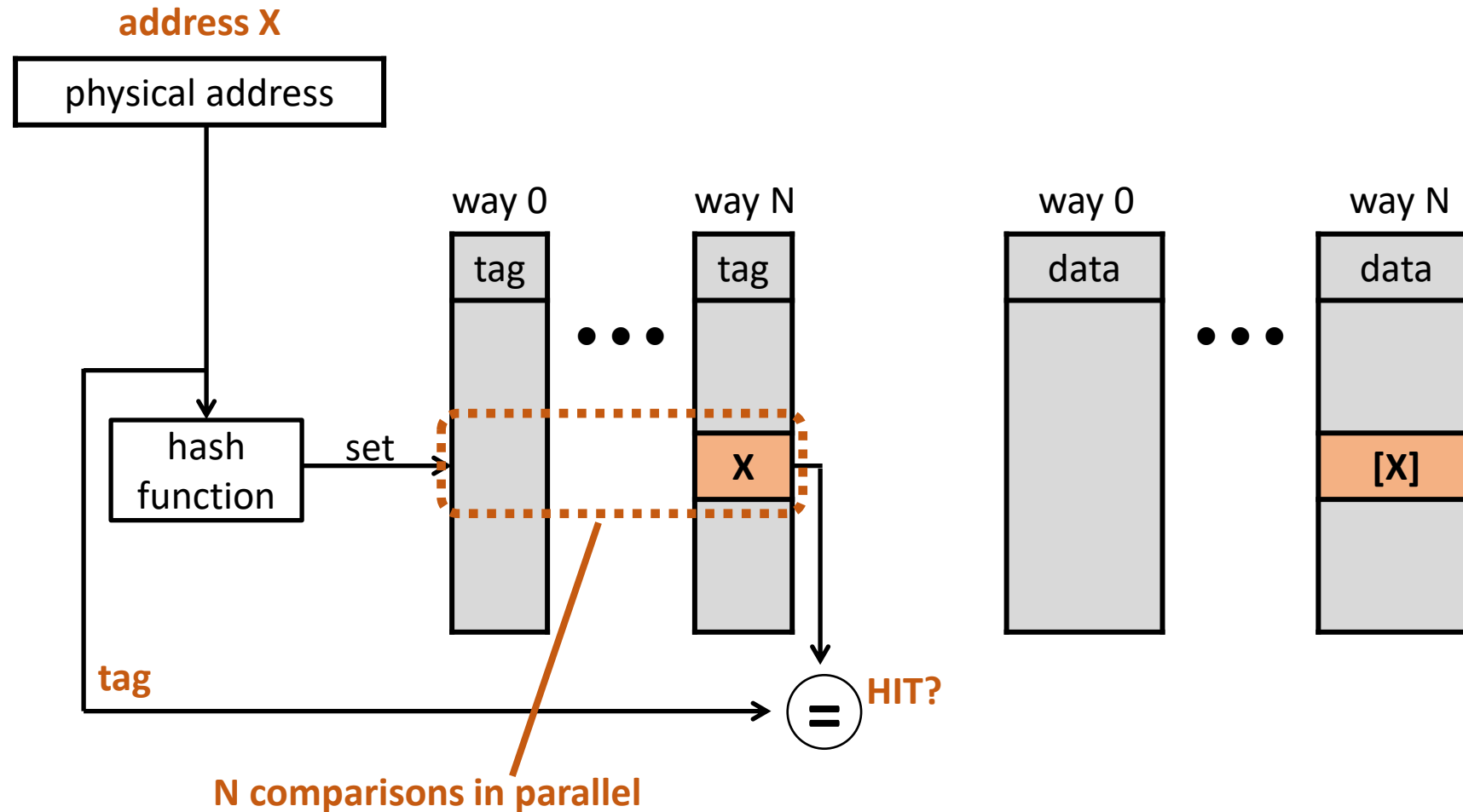
Cache Architecture – Placement



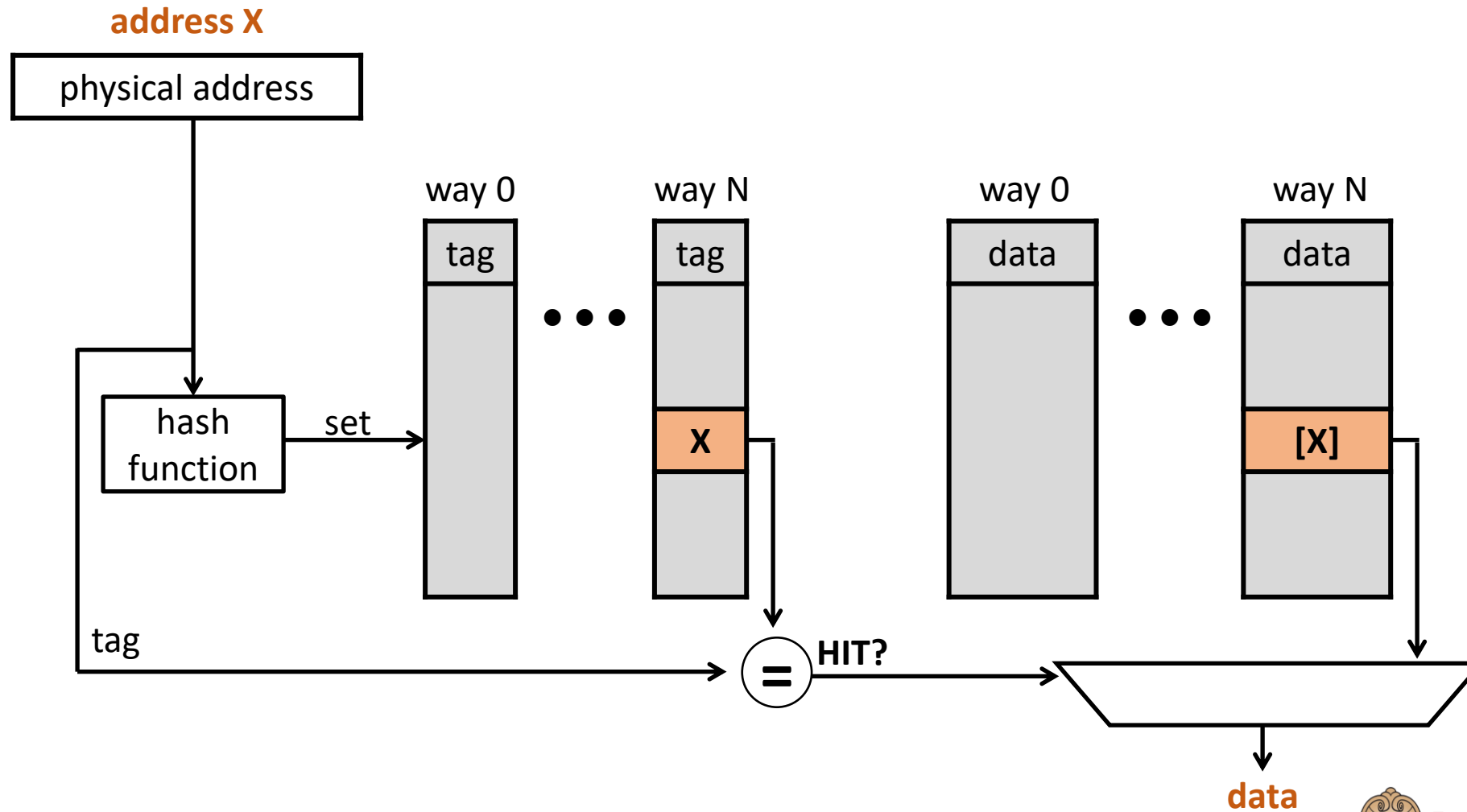
Cache Architecture – Placement



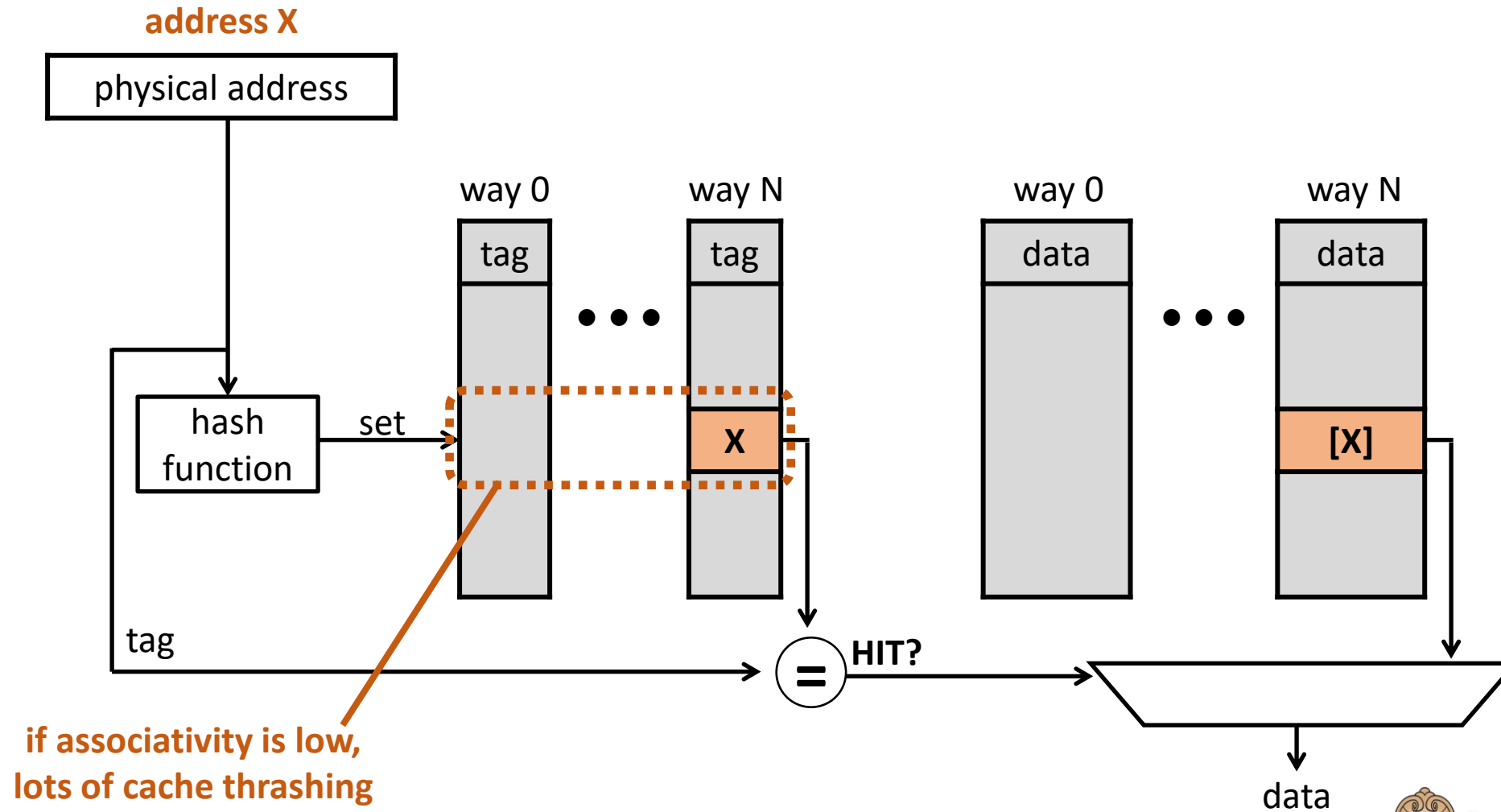
Cache Architecture – Placement



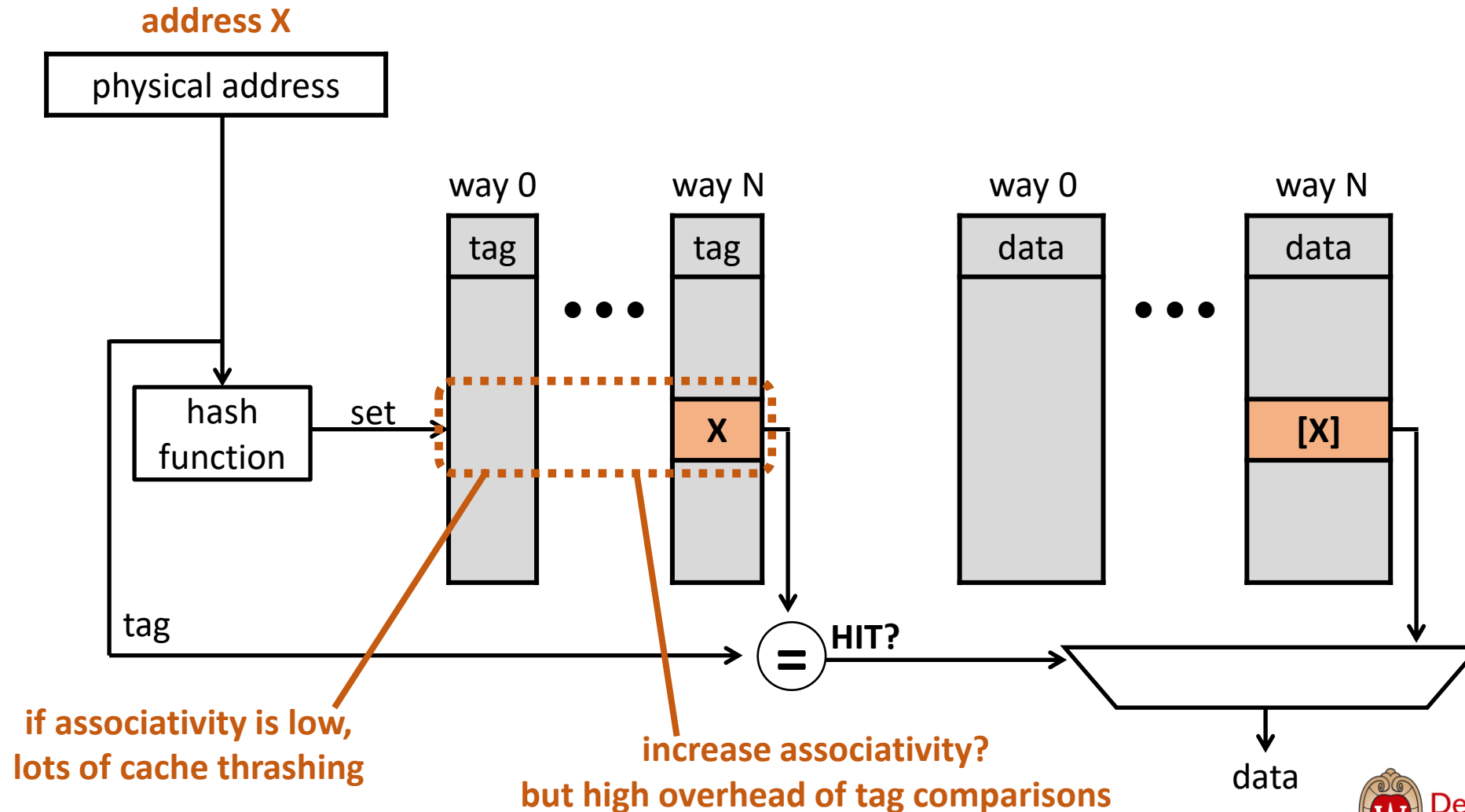
Cache Architecture – Placement



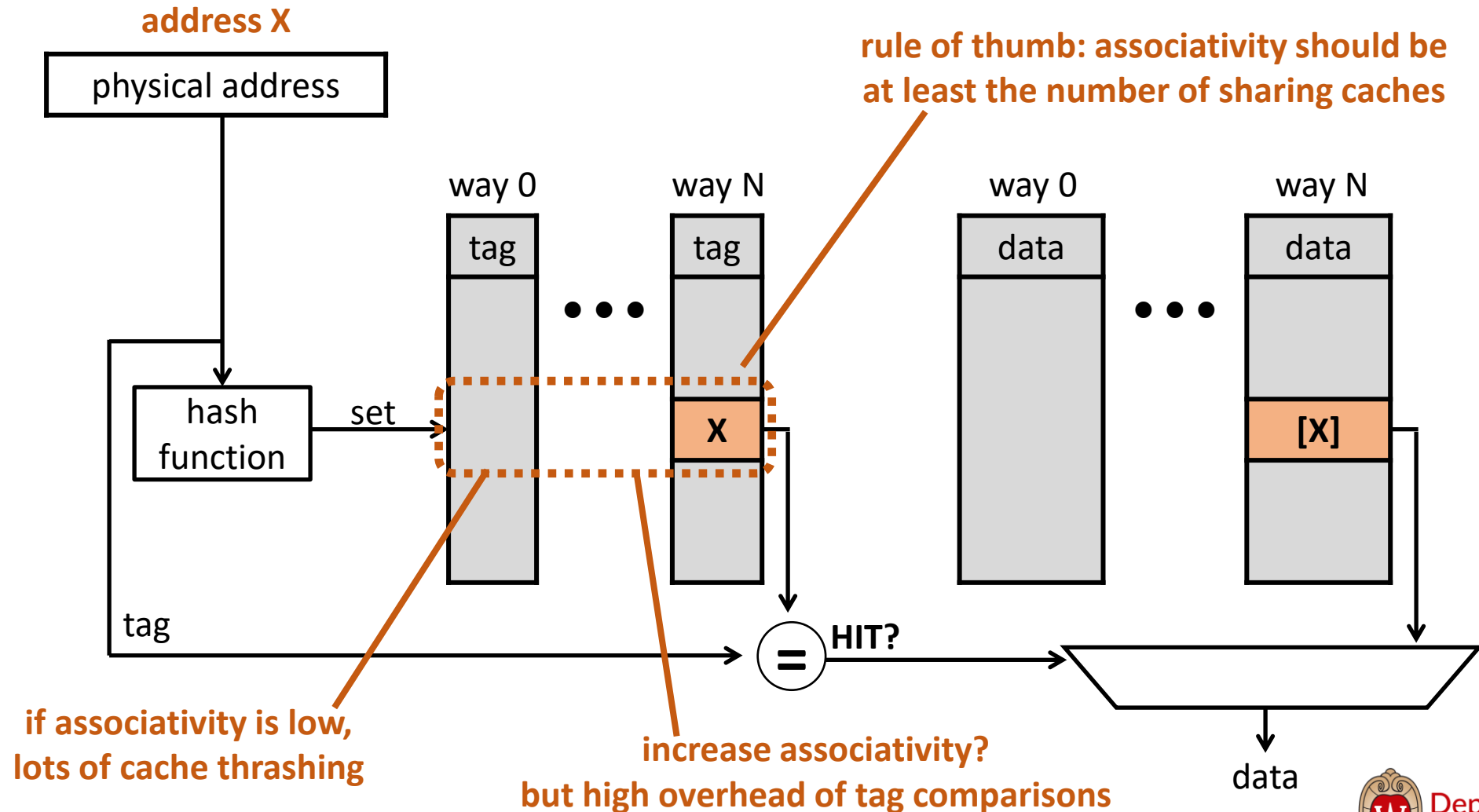
Cache Architecture – Placement



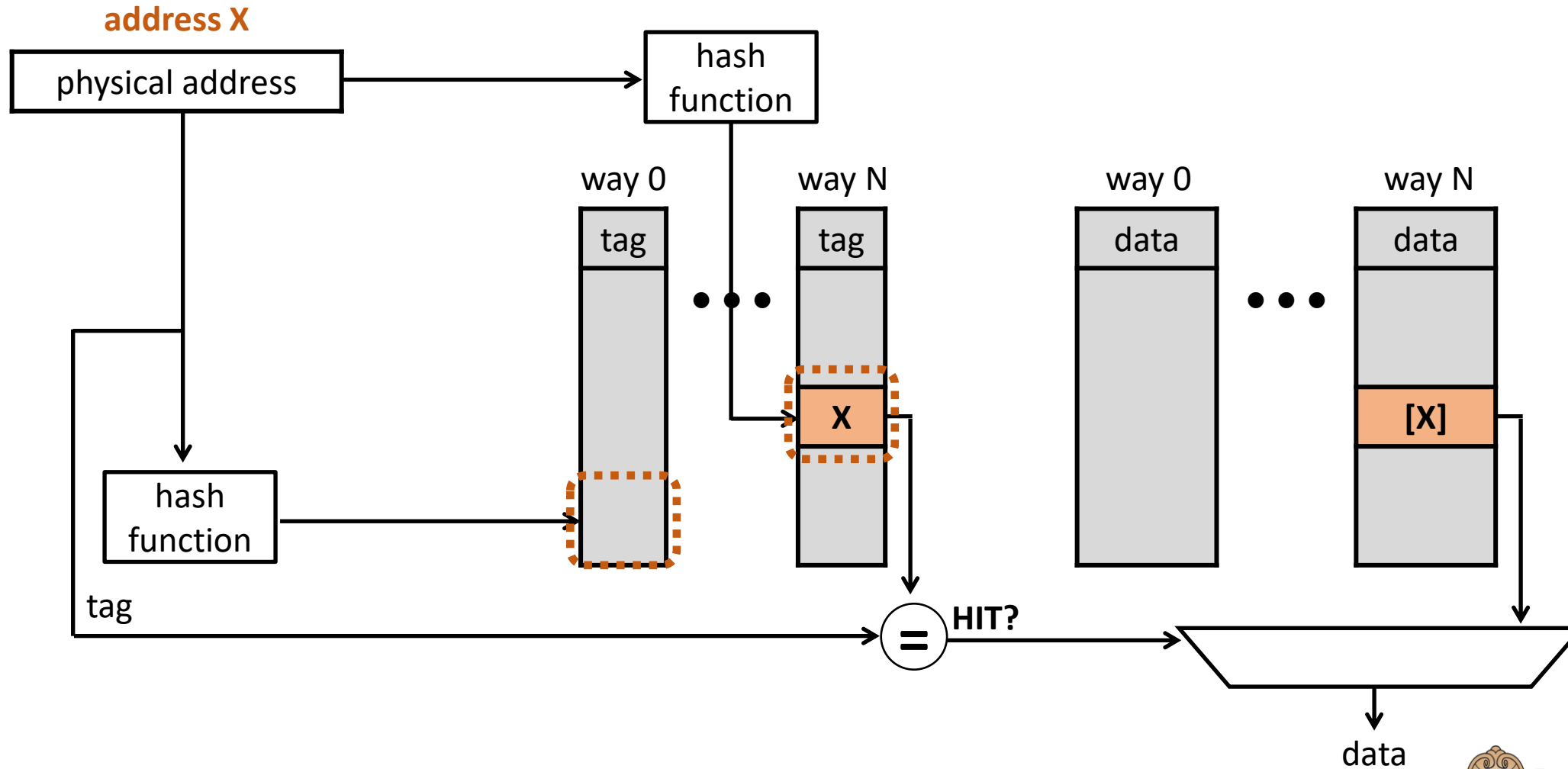
Cache Architecture – Placement



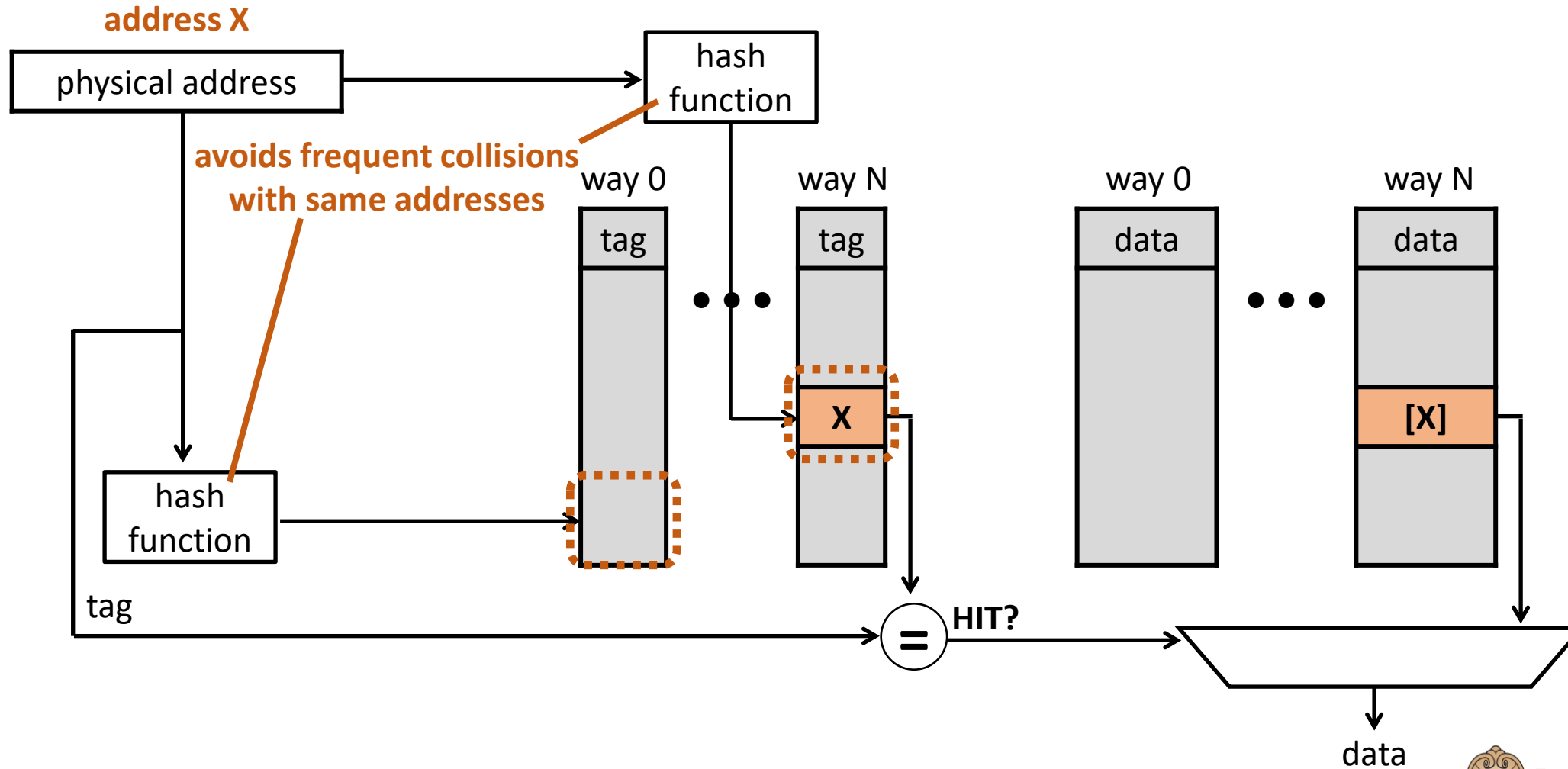
Cache Architecture – Placement



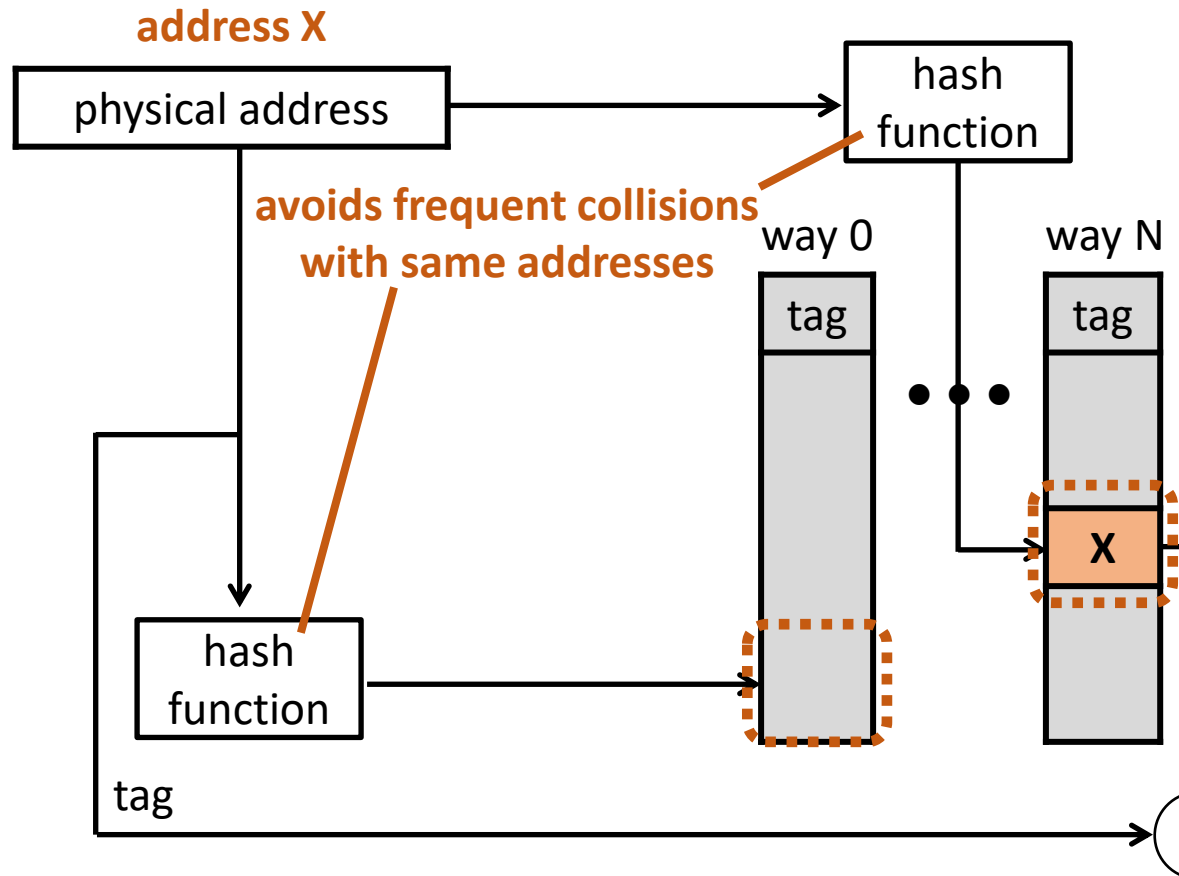
Cache Architecture – Skewed-Associative Cache



Cache Architecture – Skewed-Associative Cache

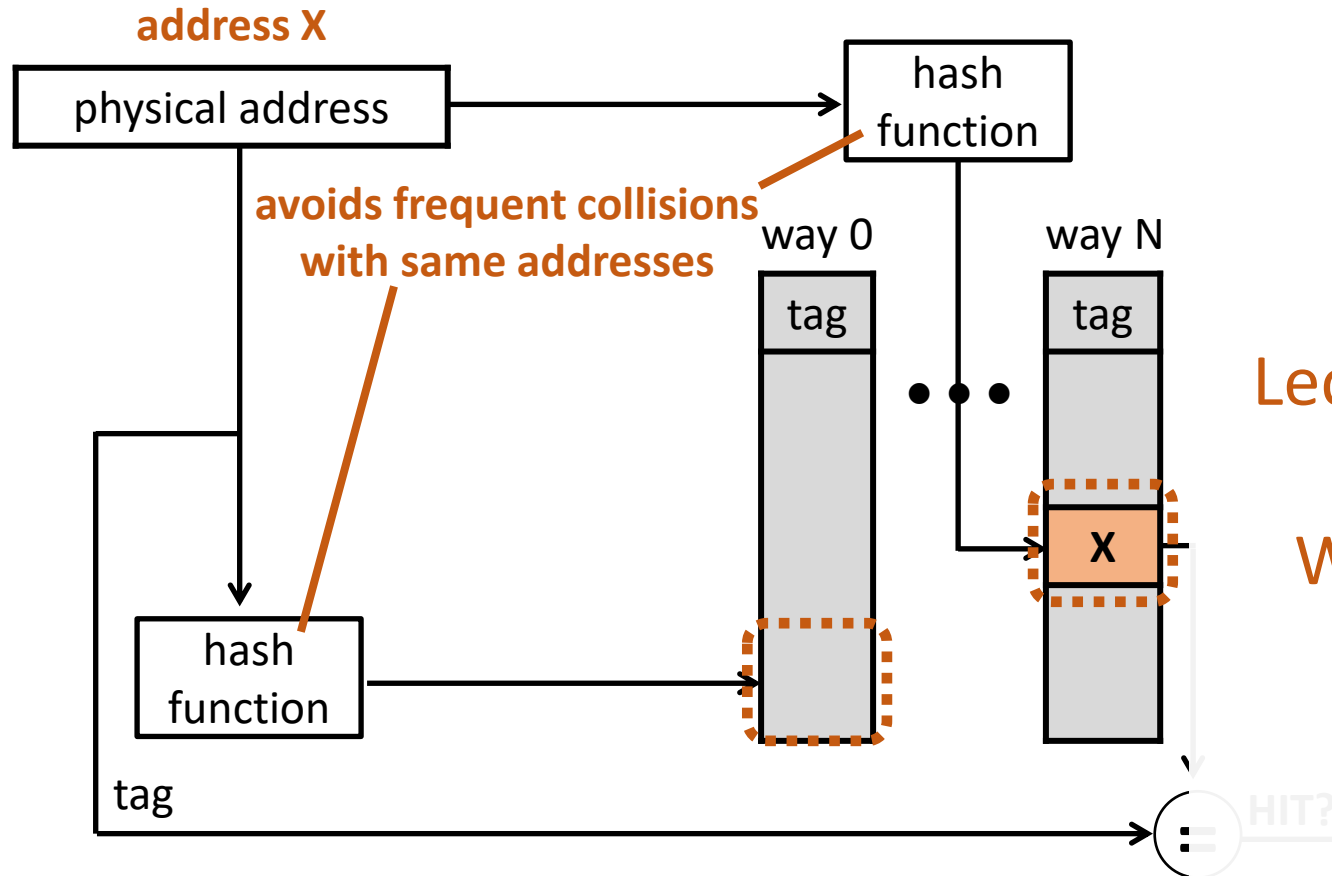


Cache Architecture – Skewed-Associative Cache



An analogy:
Tightly packed course schedule.
Lecture section for two courses collide
at MWF 1pm-2:15pm.

Cache Architecture – Skewed-Associative Cache



An analogy:

Tightly packed course schedule.
Lecture section for two courses collide
at MWF 1pm-2:15pm.

With skewed schedule, courses get:

- 1) M 1pm, W 3pm, F 10am
- 2) M 1pm, W 11am, F 4pm

Cache Architecture – Replacement

Since cache capacity is finite, decide which blocks are least likely to be accessed again soon and evict them.

FIFO (first-in-first-out): Track age using simple timestamps.

LRU (least recently used): Maintain stack of last access; complex at high associativity.

PseudoLRU: Maintain stack of last access at coarse granularity in the form of a tree.

NMRU (not most recently used): Track most recently used; simple LRU approximation.

Cache Architecture – Write Policy

Since multiple copies of a block can exist at multiple levels of the memory hierarchy, correctly propagate any modified values to all levels.

Write-Allocate: Insert the block (and its modifications) in the cache.

Write-No-Allocate: Don't occupy cache space for the block.

Write-Through: Propagate directly to the next level on every store (typically with write-no-allocate).

Write-Back: Propagate to the next level on eviction (typically with write-allocate).

Cache Misses

Cold (Compulsory) Miss:

First ever access to a given block of memory.

Capacity Miss:

Working set exceeds cache capacity.

Conflict Miss:

Placement restrictions cause useful blocks to be evicted.

Cache Misses

- Reduce **capacity** misses?
 - Increase cache capacity
 - Improve program temporal locality
- Reduce **conflict** misses?
 - Increase associativity
 - Improve hash function
 - Improve replacement policy
- Reduce **compulsory** misses?
 - Increase block size
 - Implement prefetching
 - Improve program spatial locality

Cache Misses

You are given a single-threaded application benchmark and a cache simulator, but you cannot modify either. The simulator outputs the total number of cache misses in the benchmark. As inputs to the simulator, you can configure the cache size (4B to infinite), the line size (4B to 128B) and the associativity (1-way to fully). You are asked to estimate how many **capacity misses** there would be if you were to use a 4-way 16KB cache with 64B lines. Describe how you would use the simulator to do this.

Cache Misses

You are given a single-threaded application benchmark and a cache simulator, but you cannot modify either. The simulator outputs the total number of cache misses in the benchmark. As inputs to the simulator, you can configure the cache size (4B to infinite), the line size (4B to 128B) and the associativity (1-way to fully). You are asked to estimate how many **capacity misses** there would be if you were to use a 4-way 16KB cache with 64B lines. Describe how you would use the simulator to do this.

Cold misses = misses (64B lines, infinite cache size)

**Capacity misses = misses (64B lines, 16KB cache size, fully-associative)
– cold misses**

Average Memory Access Time (AMAT)

Average time it takes between when a processor core issues a request (to the cache/memory hierarchy) for a block and when the block is received by the processor.

$$AMAT_{L1, MEM} = t_{hit, L1} + \%_{miss, L1} \times (t_{MEM})$$

Average Memory Access Time (AMAT)

Average time it takes between when a processor core issues a request (to the cache/memory hierarchy) for a block and when the block is received by the processor.

$$AMAT_{L1, MEM} = t_{hit, L1} + \%_{miss, L1} \times (t_{MEM})$$

$$AMAT_{L1, L2, MEM} = t_{hit, L1} + \%_{miss, L1} \times (t_{hit, L2} + \%_{miss, L2} \times (t_{MEM}))$$

Average Memory Access Time (AMAT)

Average time it takes between when a processor core issues a request (to the cache/memory hierarchy) for a block and when the block is received by the processor.

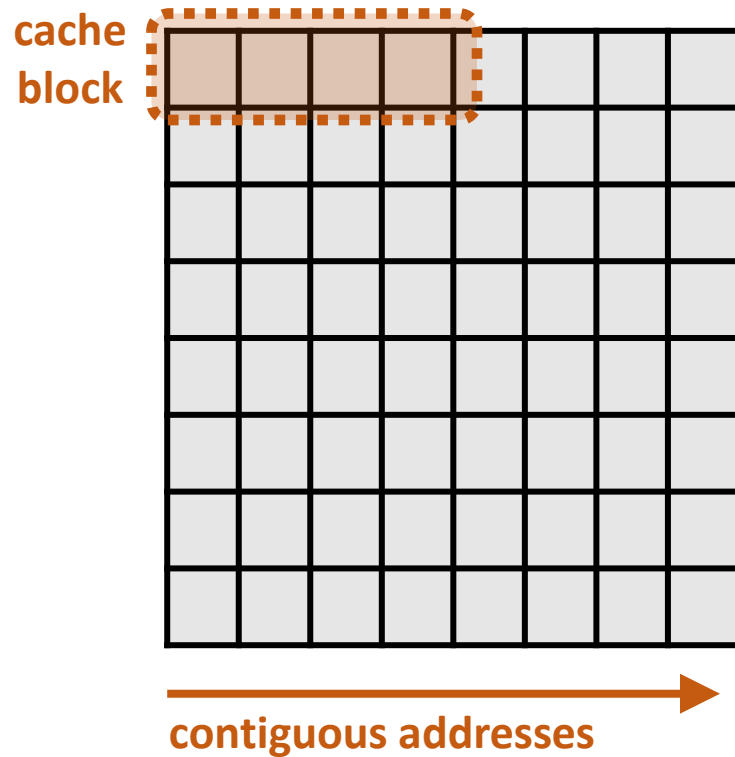
$$AMAT_{L1, MEM} = t_{hit, L1} + \%_{miss, L1} \times (t_{MEM})$$

$$AMAT_{L1, L2, MEM} = t_{hit, L1} + \%_{miss, L1} \times (t_{hit, L2} + \%_{miss, L2} \times (t_{MEM}))$$

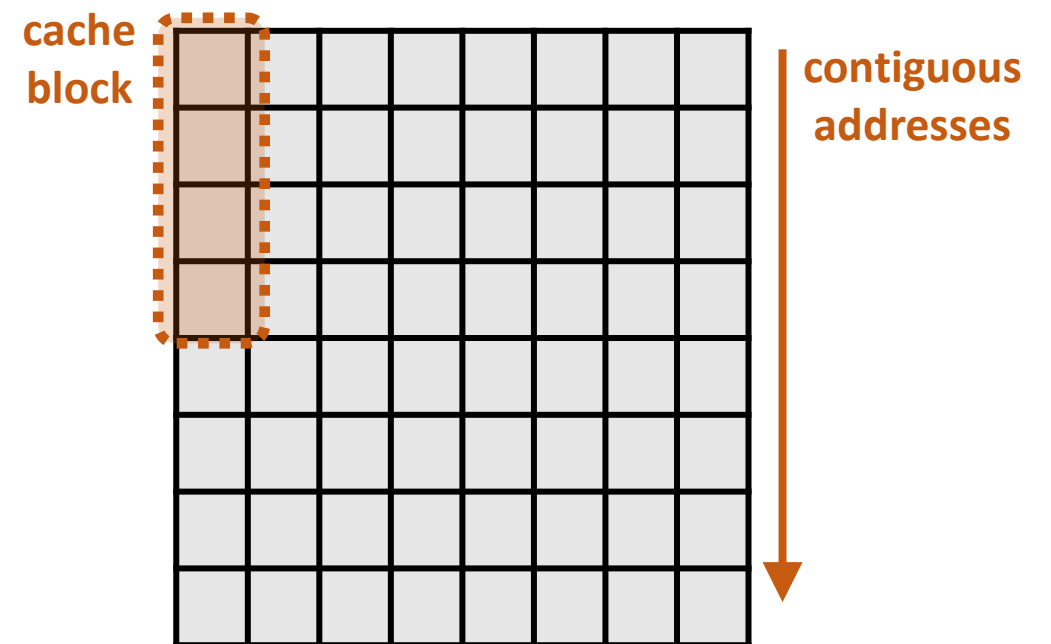
$$AMAT_{L1, L2, \dots, MEM} = t_{hit, L1} + \%_{miss, L1} \times (t_{hit, L2} + \%_{miss, L2} \times (t_{hit, \dots} + \%_{miss, \dots} \times (t_{MEM})))$$

Memory Ordering

Row-Major Order



Column-Major Order



False Sharing

Independent threads read/write unique data elements in the same block.

False Sharing

Independent threads read/write unique data elements in the same block.

```
int a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int i = 0; i < N; i++)
    for(int j = 0; j < M; j++)
        a[j][i] = ...;
```

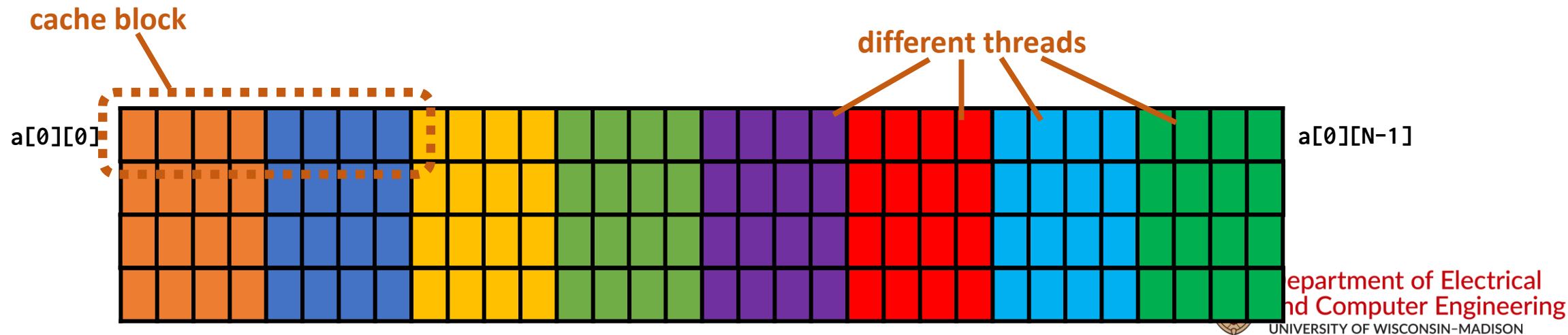
What if 8 threads?

False Sharing

Independent threads read/write unique data elements in the same block.

```
int a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int i = 0; i < N; i++)
    for(int j = 0; j < M; j++)
        a[j][i] = ...;
```

What if 8 threads?

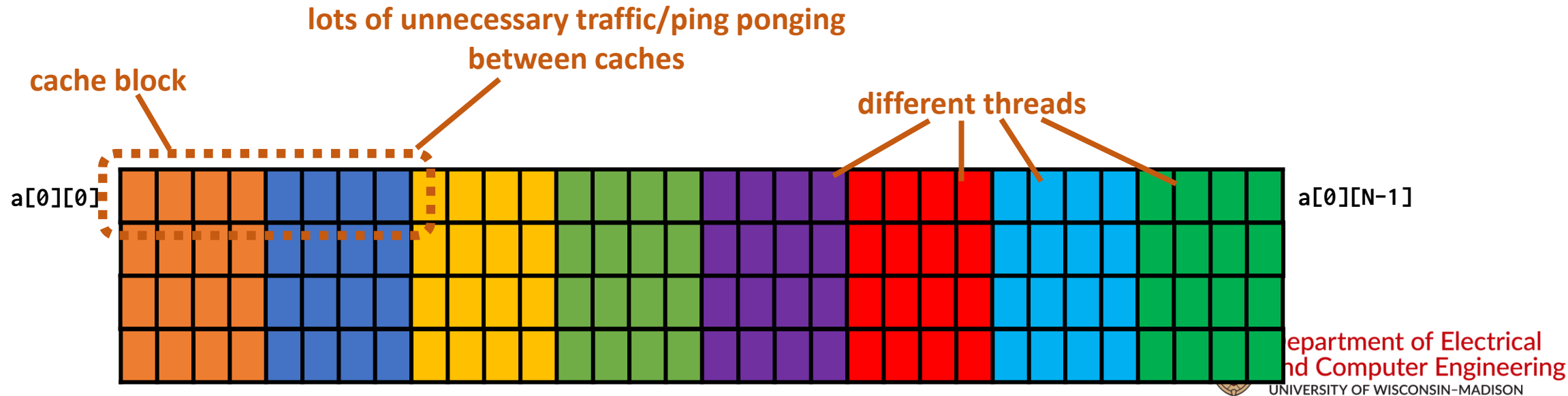


False Sharing

Independent threads read/write unique data elements in the same block.

```
int a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int i = 0; i < N; i++)
    for(int j = 0; j < M; j++)
        a[j][i] = ...;
```

What if 8 threads?



False Sharing

Independent threads read/write unique data elements in the same block.

```
int a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int i = 0; i < N; i++)
    for(int j = 0; j < M; j++)
        a[j][i] = ...;
```

False Sharing

Independent threads read/write unique data elements in the same block.

```
int a[M][N]; // with 32-byte blocks
```

```
#pragma omp parallel for
```

```
for(int j = 0; j < M; j++)
```

```
    for(int i = 0; i < N; i++)
```

```
        a[j][i] = ...;
```

if possible, improve spatial locality

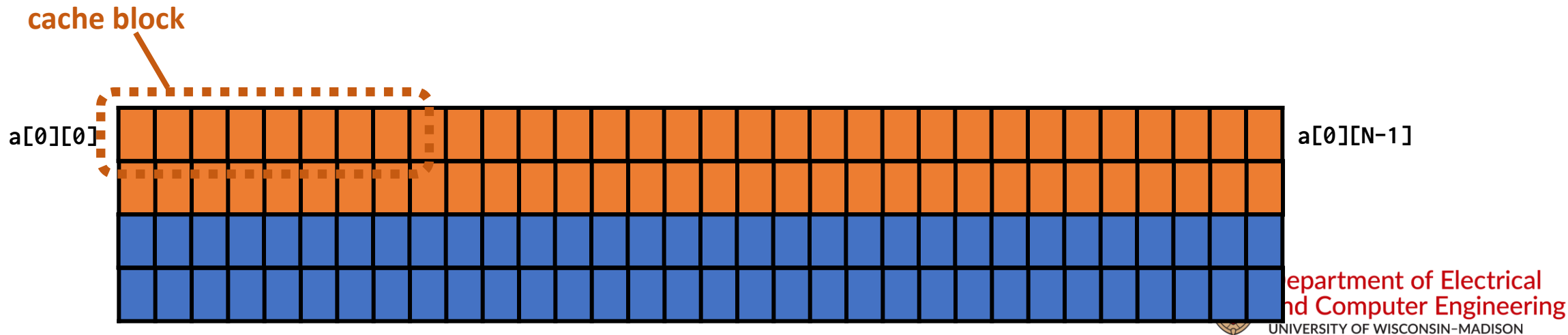


False Sharing

Independent threads read/write unique data elements in the same block.

```
int a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int j = 0; j < M; j++)
    for(int i = 0; i < N; i++)
        a[j][i] = ...;
```

if possible, improve spatial locality



False Sharing

Independent threads read/write unique data elements in the same block.

```
int a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int i = 0; i < N; i++)
    for(int j = 0; j < M; j++)
        a[j][i] = ...;
```

False Sharing

Independent threads read/write unique data elements in the same block.

```
struct{int data; char pad[4];} a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int i = 0; i < N; i++)
    for(int j = 0; j < M; j++)
        a[j][i].data = ...;
```

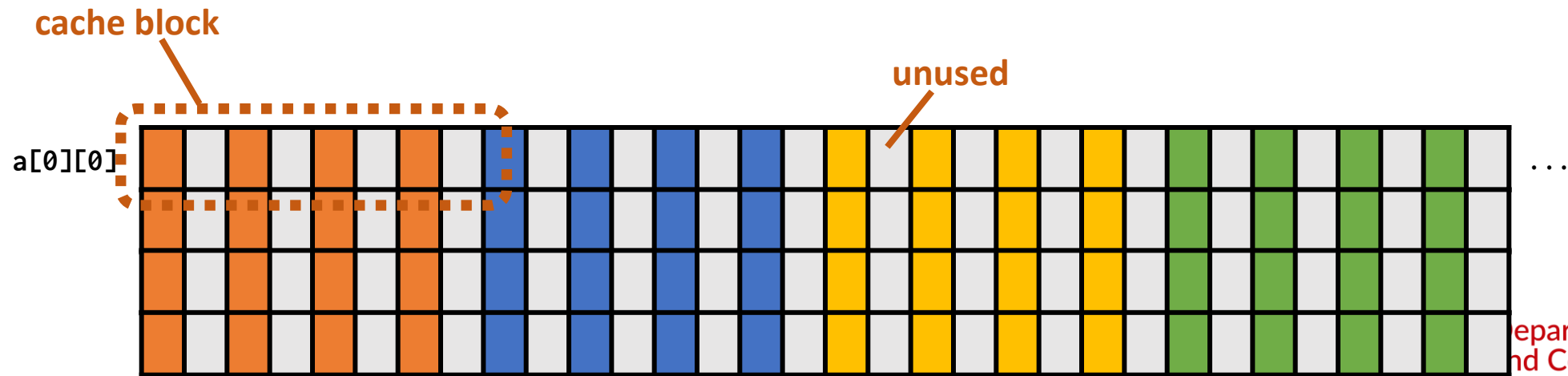
if need be, add padding

False Sharing

Independent threads read/write unique data elements in the same block.

```
struct {int data; char pad[4];} a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int i = 0; i < N; i++)
    for(int j = 0; j < M; j++)
        a[j][i].data = ...;
```

if need be, add padding



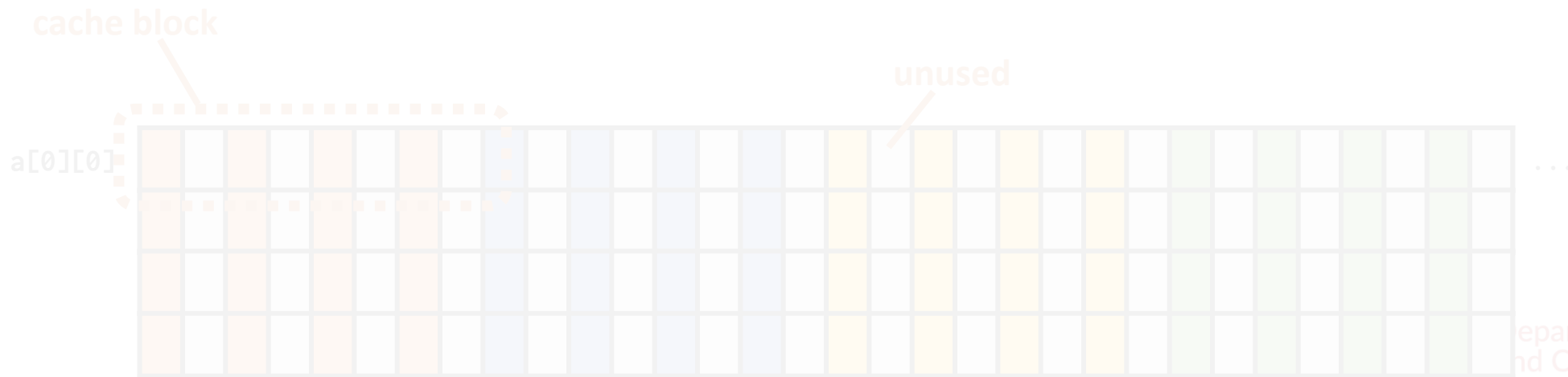
False Sharing

Independent threads read/write unique data elements in the same block.

```
struct {int data; char pad[4];} a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int i = 0; i < N; i++)
```

An analogy:

One sibling only likes the pizza crust; the other only likes the cheesy part. They never eat the same portions of the slice, but it's awkward to eat it together.



False Sharing

Independent threads read/write unique data elements in the same block.

```
struct {int data; char pad[4];} a[M][N]; // with 32-byte blocks
#pragma omp parallel for
for(int i = 0; i < N; i++)
```

An analogy:

One sibling only likes the pizza crust; the other only likes the cheesy part. They never eat the same portions of the slice, but it's awkward to eat it together.

- 1) Cut the pizza in crust-major order instead of slice-major order, or
- 2) Add excess dough in between the crust and the rest of the slice.

